

Trabajo Final: Inferencia Estadística sobre League of Legends

26 de mayo de 2026

Nicolás Graullera Villén

2026-05-26

Table of contents

1	Abstract	3
2	Introducción	3
3	Planteamiento del problema	4
4	Estado de la cuestión	5
4.1	Predicción del resultado y señales in-game	6
4.2	Ventajas estructurales: selección de lado	6
4.3	Rol, visión y arquetipos de campeón	6
4.4	Diferencias regionales: EUW frente a KR	7
4.5	De la predicción a la inferencia clásica	7
4.6	Síntesis e implicaciones para el trabajo	7
5	Datos y metodología general	7
5.1	Fuente de datos	7
5.2	Proceso de construcción del dataset	8
5.3	Variables de interés y uso en el trabajo	9
5.4	Preparación de datos	9
6	Bloque 1: Inferencia Paramétrica (50%)	11
6.1	Contraste 1: Diferencia de medias (oro al minuto 15)	11
6.1.1	Planteamiento y contexto	11
6.1.2	Hipótesis inicial	12
6.1.3	Preparación de la muestra	12
6.1.4	Verificación de supuestos	14
6.1.5	Planteamiento del contraste	15
6.1.6	Estadístico de contraste e intervalo de confianza	15
6.1.7	Contraste complementario: Prueba de rangos con signo de Wilcoxon	16
6.1.8	Contraste inferencial e intervalo de confianza	16
6.2	Contraste 2: Proporción (ventaja del lado azul)	18
6.2.1	Planteamiento y contexto	18

6.2.2	Hipótesis inicial	18
6.2.3	Preparación de la muestra	19
6.2.4	Verificación de supuestos	21
6.2.5	Planteamiento del contraste	22
6.2.6	Estadístico de contraste e intervalo de confianza	22
6.2.7	Contraste complementario: Test de signos	22
6.2.8	Contraste inferencial e intervalo de confianza	23
6.3	Contraste 3: Diferencia de medias entre roles (ADC vs Mid)	24
6.3.1	Planteamiento y contexto	24
6.3.2	Hipótesis inicial	24
6.3.3	Preparación de la muestra	25
6.3.4	Verificación de supuestos	27
6.3.5	Planteamiento del contraste	28
6.3.6	Estadístico de contraste e intervalo de confianza	28
6.3.7	Contraste complementario: Prueba U de Mann-Whitney	29
6.3.8	Contraste inferencial e intervalo de confianza	29
6.3.9	Análisis complementario: sensibilidad ante dependencia intra-partida	31
6.4	Conclusión del Bloque 1	36
7	Bloque 2: Métodos no paramétricos (30%)	36
7.1	Contraste 1: Bondad de ajuste binomial negativa (kills por equipo)	37
7.1.1	Planteamiento y contexto	37
7.1.2	Hipótesis inicial	37
7.1.3	Preparación de la muestra	38
7.1.4	Verificación de supuestos	41
7.1.5	Planteamiento del contraste	43
7.1.6	Estadístico de contraste	43
7.1.7	Contraste inferencial	44
7.2	Contraste 2: Independencia entre clase de campeón y resultado	46
7.2.1	Planteamiento y contexto	46
7.2.2	Hipótesis inicial	46
7.2.3	Preparación de la muestra	47
7.2.4	Verificación de supuestos	49
7.2.5	Planteamiento del contraste	50
7.2.6	Estadístico de contraste	50
7.2.7	Contraste inferencial	50
7.2.8	Contraste complementario: sensibilidad ante dependencia intra-partida	51
7.3	Contraste 3: Mann-Whitney (daño por minuto entre regiones)	55
7.3.1	Planteamiento y contexto	55
7.3.2	Hipótesis inicial	55
7.3.3	Preparación de la muestra	56
7.3.4	Verificación de supuestos	58
7.3.5	Planteamiento del contraste	58
7.3.6	Estadístico de contraste	58
7.3.7	Contrastes complementarios paramétricos: t de Student y t de Welch	59
7.4	Conclusión del Bloque 2	61

8	Bloque 3: Series Temporales (20%)	62
8.1	Serie temporal: duración media quincenal de las partidas	62
8.1.1	Planteamiento y contexto	62
8.1.2	Hipótesis inicial	62
8.1.3	Preparación de la serie temporal	62
8.1.4	Interpretación	65
8.1.5	Modelo ARIMA: ajuste práctico y predicción	66
8.1.6	Contraste de estacionariedad: Dickey-Fuller aumentado	66
9	Conclusiones	75
10	Bibliografía	76

1 Abstract

Este trabajo aplica técnicas de inferencia estadística a una muestra de 2.000 partidas rankeadas de *League of Legends*, obtenidas mediante la API oficial de Riot Games. El objetivo principal es comprobar con datos algunas ideas habituales dentro de la comunidad, como la importancia del oro temprano, la posible ventaja del lado azul, las diferencias entre roles, clases de campeón y regiones, y la evolución temporal de la duración de las partidas. Para ello, el análisis se organiza en tres bloques: contrastes paramétricos, contrastes no paramétricos y un modelo de serie temporal.

En el primer bloque se emplean contrastes de medias y proporciones para estudiar la diferencia de oro al minuto 15 entre equipos ganadores y perdedores, la proporción de victorias del lado azul y el daño infligido por los roles ADC y Mid. En el segundo bloque se aplican métodos no paramétricos, incluyendo bondad de ajuste, independencia mediante chi-cuadrado y la prueba U de Mann-Whitney. Por último, en el tercer bloque se analiza la duración media de las partidas a lo largo del tiempo mediante un modelo ARIMA.

Los resultados muestran una evidencia clara a favor de la importancia del oro temprano: los equipos ganadores presentan, de media, mayor oro al minuto 15. Sin embargo, no se encuentra evidencia suficiente para afirmar una ventaja significativa del lado azul ni una diferencia práctica relevante entre ADC y Mid en daño total infligido. En los métodos no paramétricos, las *kills* por equipo no se ajustan adecuadamente a una binomial negativa, no se observa dependencia relevante entre clase de campeón y resultado, y aparece una diferencia regional en *damage_per_min*, aunque con tamaño del efecto pequeño. Finalmente, la serie temporal de duración media se comporta de forma estacionaria y el modelo ARIMA seleccionado no apunta a cambios bruscos, sino a una continuidad en torno a los 29-30 minutos.

Palabras clave: inferencia estadística, *League of Legends*, contrastes de hipótesis, métodos no paramétricos, series temporales, ARIMA.

2 Introducción

League of Legends (LoL) es un videojuego (Multiplayer Online Battle Arena) desarrollado y publicado por Riot Games. En cada partida se enfrentan dos equipos de 5 jugadores cada uno.

El objetivo final es destruir el nexo (la base) del equipo enemigo. Fue uno de los videojuegos pioneros en el desarrollo de la escena de los Esports, y actualmente se realizan competiciones oficiales por todo el mundo.

Personalmente, juego y veo partidas de *League of Legends* desde hace muchos años. He presenciado cómo se ha ido desarrollando la disciplina del análisis del videojuego, tanto teóricamente y a través del estudio y análisis humano de las partidas, como a través de los datos. Esta ha sido mi motivación principal para realizar este trabajo.

Trabajos previos sobre predicción de resultados mediante aprendizaje automático han demostrado que existen variables cuantificables y robustas que pueden ser usadas para predecir el resultado de las partidas. Quiero dejar claro que este trabajo no busca la predicción, sino aplicar los conocimientos adquiridos en la asignatura para comprender el peso real de estas variables y su significación.

Cada partida registra información detallada de todas las variables que influyen en el resultado final. Esta cantidad de datos es accesible mediante la API oficial de Riot Games, y convierte al juego en un caso de estudio adecuado para aplicar técnicas de inferencia estadística y analizar si existen diferencias y asociaciones significativas entre estas variables y el resultado de la partida.

Desde la comunidad y los jugadores, se suelen describir patrones (por ejemplo, la ventaja de lado azul o impacto del control de visión), sin fundamento, únicamente teniendo en cuenta la intuición y la experiencia. El enfoque de esta memoria es ser capaz de verificar estas afirmaciones, utilizando los procedimientos de inferencia estadística estudiados en la asignatura.

Durante el desarrollo del trabajo, más concretamente en cada sección específica, se intentará explicar las preguntas que se plantean, y las variables que se seleccionan para responderlas, con el objetivo de que un lector ajeno al tema pueda entender el trabajo y las conclusiones.

3 Planteamiento del problema

En *League of Legends*, el resultado de una partida depende del oro, los objetivos, los asesinatos, las composiciones de cada equipo, etc. En la práctica, se asume que ciertas señales, como la ventaja de oro temprana, la ventaja de lado, influyen en la probabilidad de victoria, pero estas creencias no siempre se basan en datos.

Preguntas de investigación:

1. ¿El oro promedio al minuto 15 del equipo ganador es mayor que el del equipo perdedor en la misma partida?
2. ¿La probabilidad de ganar desde el lado azul difiere de 0,5?
3. ¿Hay diferencias de daño infligido entre roles?
4. ¿Las **kills** de cada equipo siguen alguna distribución?
5. ¿Existen clases de campeones más efectivas que otras?
6. ¿Las partidas en KR son más frenéticas que en EUW?
7. ¿Cómo evoluciona la duración media de las partidas a lo largo del periodo observado?

Objetivo general: evaluar, mediante técnicas de inferencia estadística, si existen diferencias y asociaciones significativas entre variables de rendimiento en *League of Legends* y el resultado de la partida.

Bloque 1 — Inferencia paramétrica

Cuantificar la diferencia de oro al minuto 15 entre equipos ganadores y perdedores, con intervalo de confianza. Contrastar si la proporción de victorias del lado azul es superior a 0,5. Comparar el daño infligido entre los roles ADC y Mid mediante contraste de medias.

Bloque 2 — Métodos no paramétricos

Evaluar si los asesinatos por equipo se ajustan a una distribución binomial negativa (bondad de ajuste). Analizar la dependencia entre la clase de campeón y el resultado de la partida (chi-cuadrado). Comparar la distribución de daño por minuto entre las regiones EUW y KR (Mann-Whitney).

Bloque 3 — Series temporales

Analizar la evolución temporal de la duración media de las partidas y ajustar un modelo ARIMA exploratorio.

Variables y unidad de análisis:

Se emplean variables in-game obtenidas de la API Match-V5: `gold_15` (economía temprana), `damage_dealt` y `damage_per_min` (producción de combate), `kills` (eventos de eliminación), `wards_placed` (control de visión), `side` y `win` (contexto y resultado), `role` y `champion_class` (perfil funcional), y `region` (EUW vs KR). La unidad de análisis será el jugador-partida para contrastes por rol, el equipo-partida cuando la pregunta lo exija (oro agregado, ventaja azul, kills de equipo) y la partida completa para la serie temporal del Bloque 3.

Alcance:

El estudio se limita a partidas rankeadas Solo/Duo (`queue_id = 420`) en Summoner's Rift, con una muestra objetivo de 2.000 partidas balanceadas entre EUW y KR, extraídas mediante un pipeline reproducible desde la API oficial de Riot Games. No se persigue construir un modelo predictivo global, sino responder preguntas concretas mediante contrastes de hipótesis, intervalos de confianza y análisis temporal exploratorio.

Limitaciones:

- La muestra depende de jugadores semilla y de la disponibilidad de la API;
- Puede existir dependencia entre observaciones de una misma partida;
- Los efectos pueden variar entre parches y metas;
- Los resultados describen asociaciones en la muestra, no relaciones causales estrictas.

4 Estado de la cuestión

El análisis de datos aplicado a *League of Legends* ha evolucionado desde descripciones básicas de partidas o rendimiento de jugadores en páginas web, hasta estudios de predicción del resultado de las partidas. Sin embargo, la mayoría de los artículos o trabajos proceden de modelos de aprendizaje automático, con escasa interpretación, y que priorizan la precisión predictiva sobre el entendimiento de los datos del juego.

4.1 Predicción del resultado y señales in-game

Una de las líneas de investigación más relevante es la predicción del resultado de la partida. Lin (2016), mediante el uso de *gradient boosted trees*, demostró que la capacidad de predicción aumenta considerablemente cuando se incorporan estadísticas *in-game* (daño total de cada jugador, diferencia de oro entre dos jugadores, *KDA* (Kills, Deaths, Assists)) frente a modelos que utilizan únicamente información pre-partida. El autor afirma: “We find that using only prematch knowledge (champions, masteries, roles, spells) from the very start is only a weak predictor of match outcome but using in-game statistics, the model becomes a strong predictor.”

Posteriormente, Do et al. (2021) profundizaron en el conjunto de datos pre-partida utilizando una red neuronal profunda, y demostraron que la experiencia acumulada de un jugador con un campeón es una variable significativa para predecir el resultado de la partida. Los autores alcanzaron una precisión del 75,1 % en la predicción del resultado una vez seleccionados los campeones, es decir, antes del comienzo de la partida.

Ambos trabajos presentan limitaciones desde una perspectiva explicativa, ya que priorizan el rendimiento predictivo sobre la interpretación estadística de las variables empleadas.

4.2 Ventajas estructurales: selección de lado

Uno de los temas más tratados en la comunidad competitiva es la ventaja atribuida al **lado azul** frente al rojo. En partidas de alto nivel se discute con frecuencia si la disposición del mapa, el acceso a objetivos o el orden del *draft* (selección de campeones) aportan ventaja. Jack J. (2022), en un análisis técnico publicado en *The Esports Analyst Club*, habla sobre la relevancia de la selección de lado y concluye que, si bien la ventaja existe en muchos contextos, su magnitud es moderada y depende del parche, la región y el nivel competitivo considerado.

4.3 Rol, visión y arquetipos de campeón

El LoL organiza a los jugadores en 5 **roles** (Top, Jungle, Mid, ADC, Support) con funciones diferenciadas. Cada uno de estos roles se ha ido desarrollando con el paso del tiempo, especializándose más en ciertas funciones. Por ejemplo, el rol de ADC está orientado a la obtención de recursos para realizar el mayor daño posible al equipo rival. En cambio, el rol de Support adopta una función más utilitaria, priorizando el apoyo al equipo y el control de la visión.

Paralelamente, la **clase de campeón** (tanque, mago, asesino, etc.) condiciona el estilo de combate y, en consecuencia, variables como el daño infligido o el número de asesinatos. Do et al. (2021) y Lin (2016) incorporan información de campeón en sus modelos, lo que indica que la composición importa para explicar el resultado.

Por tanto, rol y clase de campeón se tendrán en cuenta a la hora de seleccionar variables y formular los contrastes inferenciales.

4.4 Diferencias regionales: EUW frente a KR

La comparación entre regiones es otro tema recurrente en la comunidad. **Corea (KR)** suele considerarse mejor, destacando por un nivel mecánico alto y una mayor agresividad temprana, mientras que **Europa (EUW)** presenta un estilo algo más diverso en duración y ritmo de partida. Estas afirmaciones proceden a menudo de la propia experiencia de los jugadores o de medias descriptivas en páginas web, y carecen de pruebas estadísticas que las respalden.

4.5 De la predicción a la inferencia clásica

En conjunto, el estado actual de la cuestión demuestra que existen señales robustas en datos oficiales de Riot —experiencia, oro, daño, composición— relacionadas con el desenlace de las partidas. No obstante, predomina un enfoque **predictivo-algorítmico** frente a uno **inferencial-explicativo**.

Este trabajo se sitúa en ese hueco. Se plantearán contrastes transparentes y trazables alineados con los contenidos de la asignatura para entender el peso real de las variables y su significación.

4.6 Síntesis e implicaciones para el trabajo

El estado de la cuestión respalda las siguientes decisiones:

1. **Variables in-game con respaldo empírico:** `gold_15`, daño y kills aparecen en trabajos predictivos como señales relevantes. Serán utilizadas a lo largo de los diferentes contrastes.
2. **Rol, visión y clase como dimensiones funcionales:** la literatura y la práctica competitiva sugieren diferencias por rol y composición, por lo que se tendrán en cuenta a la hora de seleccionar las variables y desarrollar los contrastes.
3. **Regiones comparables pero distintas en estilo:** Por ello nuestro dataset contará con jugadores de ambas regiones.

5 Datos y metodología general

5.1 Fuente de datos

Los datos utilizados en este trabajo proceden de la API oficial de Riot Games, concretamente del endpoint Match-V5. A partir de esta API se han descargado partidas rankeadas de *League of Legends*, lo que permite trabajar tanto con variables finales de la partida como con variables tempranas, por ejemplo el oro al minuto 15.

El estudio se limita a partidas de Solo/Duo en Summoner's Rift (`queue_id = 420`). Esto se hace para mantener cierta homogeneidad: no tendría sentido mezclar partidas clasificatorias normales con otros modos de juego, porque el ritmo, la duración y el comportamiento de los jugadores pueden ser distintos.

La muestra final contiene 2.000 partidas, balanceadas entre EUW y KR. A partir de ellas se construyen dos tablas de trabajo:

- `data/matches_player_level.csv`: tabla a nivel jugador-partida, con 20.000 filas.
- `data/matches_team_level.csv`: tabla a nivel equipo-partida, con 4.000 filas.

La primera tabla se utiliza cuando la pregunta requiere información de jugadores o roles, como la comparación entre ADC y Mid. La segunda se utiliza cuando el análisis tiene sentido a nivel de equipo o partida, como la ventaja de oro, la victoria del lado azul, las `kills` por equipo o la duración media de la partida.

5.2 Proceso de construcción del dataset

El dataset no se obtiene de un repositorio estático, sino que se construye desde cero mediante la API oficial de Riot. Esto es importante porque permite que el trabajo sea reproducible: se puede volver a ejecutar la extracción, procesar los datos y obtener las mismas tablas analíticas siguiendo el mismo pipeline.

El proceso tiene cuatro fases: selección de jugadores semilla, extracción cruda, procesamiento analítico y validación de calidad.

1) Selección de semillas (puuid)

El identificador base de jugador es el `puuid` (*platform unique universal identifier*), que Riot utiliza para consultar historial de partidas de forma estable y sin depender del nombre de invocador.

Para reducir el sesgo muestral, las semillas de jugadores se seleccionan mediante muestreo estratificado por tier en cada región de estudio (EUW y KR). Los estratos utilizados son Bronze, Silver, Gold, Platinum, Emerald y Diamond. El procedimiento aplicado es:

- consulta de jugadores rankeados SoloQ (`RANKED_SOLO_5x5`) por tier/división;
- construcción de un pool de `puuid` por estrato;
- asignación de cuotas por estrato para cubrir un tamaño objetivo por región;
- selección aleatoria reproducible dentro de cada estrato (semilla fija);
- relleno con excedentes cuando un estrato no alcanza su cuota.

La salida de esta fase es `data/seed_puuids.json`, y el detalle de distribución por estrato queda registrado en `data/seed_sampling_report.json`.

2) Extracción cruda de partidas

Con las semillas definidas, se realiza extracción incremental desde Match-V5:

- recuperación de `match_id` por `puuid`;
- descarga de detalle de partida (`match`);
- descarga de cronología (`timeline`) para reconstrucción de variables temporales como `gold_15`.

La extracción incorpora control de límites de la API (rate limiting por routing value), reintentos ante errores transitorios y persistencia incremental para evitar pérdida de avance en ejecuciones largas. Los datos crudos se almacenan en `data/raw/matches/` y `data/raw/timelines/`, con metadatos en `data/raw/metadata.json`.

3) Procesamiento y normalización analítica

La transformación de JSON crudo a tablas analíticas produce:

- `data/matches_player_level.csv`, con grano jugador-partida.
- `data/matches_team_level.csv`, con grano equipo-partida.

En esta fase se aplican reglas de limpieza y derivación: filtrado por nivel rankeado, mapeo de clase de campeón (`champion_class`) y cálculo de variables derivadas como `damage_per_min`, `gold_15` y `match_duration_min`.

4) Validación de calidad

Antes del análisis inferencial, se ejecutan controles automáticos sobre:

- duplicados (`match_id + puuid`);
- presencia de nulos en columnas críticas;
- consistencia de colas;
- cobertura de regiones objetivo;
- tamaño mínimo de muestra.

El resultado se resume en `data/data_quality_report.json`, que actúa como test de la calidad del conjunto final utilizado en los contrastes.

5.3 Variables de interés y uso en el trabajo

Las variables que se han utilizado finalmente en el análisis son:

- `win`: resultado de la partida o del equipo.
- `gold_15`: oro medio del equipo al minuto 15.
- `damage_dealt`: daño total infligido por jugador.
- `damage_per_min`: daño por minuto, utilizado para comparar regiones controlando la duración.
- `kills`: asesinatos conseguidos por equipo.
- `side`: lado del mapa (`blue` o `red`).
- `role`: rol del jugador (Top, Jungle, Mid, ADC, Support).
- `champion_class`: clase de campeón.
- `region`: región de la partida (EUW o KR).
- `match_duration_min`: duración de la partida en minutos.
- `game_creation`: fecha de creación de la partida, utilizada para construir la serie temporal.

Además, en los contrastes y análisis donde la duración podía distorsionar el resultado, se filtraron las partidas de duración inferior a 5 minutos. Estas partidas suelen corresponder a *remakes* o partidas anómalas, y no representan bien una partida desarrollada con normalidad. Este filtro se aplica especialmente en los análisis de `kills`, `damage_per_min` y en la serie temporal de duración.

5.4 Preparación de datos

```
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import optimize, stats

# Configuración de matplotlib
plt.rcParams.update({"figure.dpi": 110, "font.size": 10})

df = pd.read_csv("data/matches_player_level.csv")
df_team = pd.read_csv("data/matches_team_level.csv")

# Filtro de seguridad: ranked solo/duo
df = df[df["queue_id"] == 420].copy()
df_team = df_team[df_team["queue_id"] == 420].copy()

required_cols = [
    "win", "side", "role", "gold_15", "damage_dealt",
    "damage_per_min", "kills", "wards_placed", "region",
]
missing_cols = [c for c in required_cols if c not in df.columns]
if missing_cols:
    raise ValueError(f"Faltan columnas requeridas en el dataset:
    ↪ {missing_cols}")

# Eliminamos las filas con valores nulos en las columnas requeridas
df = df.dropna(subset=required_cols)
# Eliminamos las filas con el rol "Unknown"
df = df[df["role"] != "Unknown"].copy()

ROLE_ORDER = ["Top", "Jungle", "Mid", "ADC", "Support"]
CONTINUOUS_VARS = ["gold_15", "damage_dealt", "damage_per_min", "kills",
    ↪ "wards_placed"]

print(f"Filas jugador-partida: {len(df):,}")
print(f"Partidas únicas: {df['match_id'].nunique():,}")
print(f"Filas equipo-partida: {len(df_team):,}")
df.head()
```

Filas jugador-partida: 19,988
Partidas únicas: 2,000
Filas equipo-partida: 4,000

	match_id	puuid	game_creation
0	EUW1_6996694805	RGlMVS3aRVsco8wpP44rTwppL7XN1RRiqHOyBOtwKGaSSC...	1720375117320
1	EUW1_6996694805	R8kJsnXBHd8sVUBvofeOhg91_lgGVSoVF7qi93zOA7_9ms...	1720375117320
2	EUW1_6996694805	ToJbCx_hsiVTFw76tCqiN0QPBPu0LaRCWIaIcY_A2RwHm_...	1720375117320
3	EUW1_6996694805	-COEh14H52CdSLzZhCtEdnEYtOCeJO3cVpQKlZkaY_Jqv...	1720375117320
4	EUW1_6996694805	I6sYTJbL5UXANxiaQh_jXhTX8zLN2ov1fFF5FIR3J5zKbZ...	1720375117320

Podemos ver que se han eliminado 12 filas, que en este caso corresponden a jugadores con el rol “Unknown”. Sin embargo, en relación a las partidas únicas o las filas equipo-partida, no se ven afectadas.

6 Bloque 1: Inferencia Paramétrica (50%)

En este bloque se plantean y resuelven tres contrastes de hipótesis, todos ellos acompañados de intervalos de confianza, verificación de supuestos e interpretación contextual. Se trabajará con un nivel de significación de $\alpha = 0,05$. Cuando se usen contrastes complementarios no paramétricos, se interpretarán como análisis de robustez, no como sustitutos del bloque paramétrico.

6.1 Contraste 1: Diferencia de medias (oro al minuto 15)

6.1.1 Planteamiento y contexto

Pregunta: ¿el oro promedio al minuto 15 del equipo ganador es mayor que el del equipo perdedor en la misma partida?

En *League of Legends*, el oro es el recurso central del juego: se obtiene matando campeones, destruyendo estructuras u obteniendo objetivos (dragones, barones, etc.) y se invierte en objetos que mejoran al campeón.

En la comunidad, muchas veces se tienen discusiones sobre cuánto afecta el *early game* (primeros minutos de la partida) en la victoria. Dependiendo de esto, se podrían implementar diferentes estrategias de juego.

Para responder a esta pregunta, en cada partida comparamos el oro al minuto 15 del equipo que acaba ganando con el del que acaba perdiendo. Si la ventaja temprana es realmente significativa respecto al resultado de la partida, la diferencia media entre el equipo ganador y el perdedor debería ser positiva.

Para ello utilizaremos filas de equipo-partida (`df_team`, $n = 4\,000$). En este dataframe, `gold_15` es la media del oro al minuto 15 de los cinco jugadores del equipo (no el oro total de ventaja).

Transformaremos este dataframe en pares emparejados (ganador vs perdedor) por `match_id`. Contamos con $n = 2\,000$ partidas emparejadas, con las variables `match_id`, `gold_win`, `gold_lose` y `diff` (diferencia de oro al minuto 15 entre ganador y perdedor).

6.1.2 Hipótesis inicial

Contamos con una muestra pareada formada por dos variables aleatorias:

- X : oro al minuto 15 del equipo ganador en una partida.
- Y : oro al minuto 15 del equipo perdedor en la misma partida.

Se dispone de una muestra aleatoria de $n = 2000$ observaciones pareadas (X_i, Y_i) , $i = 1, \dots, n$. A partir de ellas se construye la muestra de diferencias:

$$D_i = X_i - Y_i$$

En el código, D_i corresponde a la columna `diff` (`= gold_win - gold_lose`). Esta muestra tiene media muestral $\bar{D}$ y cuasi-desviación típica s_D (desviación típica muestral de las diferencias).

Se asume independencia entre observaciones, dado que cada par (X_i, Y_i) procede de una partida distinta.

No obstante, esta independencia debe considerarse aproximada y no estricta, ya que un mismo jugador puede aparecer en múltiples partidas de la muestra.

6.1.3 Preparación de la muestra

```
def build_gold15_pairs(team_df: pd.DataFrame) -> pd.DataFrame:
    """Empareja gold_15 ganador vs perdedor por match_id (n = n° de
    ↪ partidas)."""

    # Extraemos los ganadores y guardamos `gold_15` como `gold_win`
    winners = team_df.loc[team_df["win"] == 1, ["match_id", "gold_15"]].rename(
        columns={"gold_15": "gold_win"}
    )

    # Extraemos los perdedores y guardamos `gold_15` como `gold_lose`
    losers = team_df.loc[team_df["win"] == 0, ["match_id", "gold_15"]].rename(
        columns={"gold_15": "gold_lose"}
    )

    # Unimos los ganadores y los perdedores por `match_id` en un solo dataframe
    ↪ y
    # calculamos la diferencia de oro entre el equipo ganador y el perdedor.
    paired = winners.merge(losers, on="match_id", validate="one_to_one")
    paired["diff"] = paired["gold_win"] - paired["gold_lose"] # D_i = X_i -
    ↪ Y_i
    return paired

paired_c1 = build_gold15_pairs(df_team)
print(f"Partidas emparejadas: {len(paired_c1):,}")
paired_c1[["gold_win", "gold_lose", "diff"]].describe().round(2)
```

Partidas emparejadas: 2,000

	gold_win	gold_lose	diff
count	2000.00	2000.00	2000.00
mean	5425.60	4997.74	427.86
std	877.39	813.21	619.36
min	500.00	500.00	-1562.00
25%	5190.35	4800.95	24.75
50%	5503.60	5093.20	375.60
75%	5833.80	5380.90	806.15
max	7148.00	6588.40	2945.20

```
diff_c1 = paired_c1["diff"]
mu_d = diff_c1.mean()

# Eje x simétrico respecto a 0
x_lim = max(abs(diff_c1.min()), abs(diff_c1.max())) * 1.05

fig, ax = plt.subplots(figsize=(7.0, 3.5))

bins = np.linspace(-x_lim, x_lim, 31)
ax.hist(
    diff_c1,
    bins=bins,
    density=True,
    color="steelblue",
    edgecolor="white",
    alpha=0.9,
    label="Histograma de $D_i$",
)

x = np.linspace(-x_lim, x_lim, 300)
kde = stats.gaussian_kde(diff_c1)
ax.plot(x, kde(x), color="crimson", lw=1.8, label="Densidad estimada (KDE)",
        ↪ zorder=3)

ax.axvline(
    mu_d,
    color="darkgreen",
    ls="--",
    lw=1.2,
    label=f"$\\bar{{D}}$ = {mu_d:.0f} oro",
    zorder=3,
)

ax.set_xlim(-x_lim, x_lim)
ax.set_ylim(bottom=0)
```

```
# Eje y visible en x = 0
ax.spines["left"].set_position(("data", 0))
ax.spines["left"].set_linewidth(1.2)
ax.spines["bottom"].set_visible(True)
ax.spines["right"].set_visible(False)
ax.spines["top"].set_visible(False)
ax.set_yticks([])
ax.tick_params(axis="y", left=False, labelleft=False)
ax.xaxis.set_ticks_position("bottom")

ax.set_title("Diferencias pareadas  $D_i = X_i - Y_i$ ", fontsize=11)
ax.set_xlabel(" $D_i$  (oro)", fontsize=10)
ax.tick_params(axis="x", labelsz=9, direction="out")
ax.legend(fontsize=8, loc="upper right")

fig.tight_layout(pad=0.6)
plt.show()
```

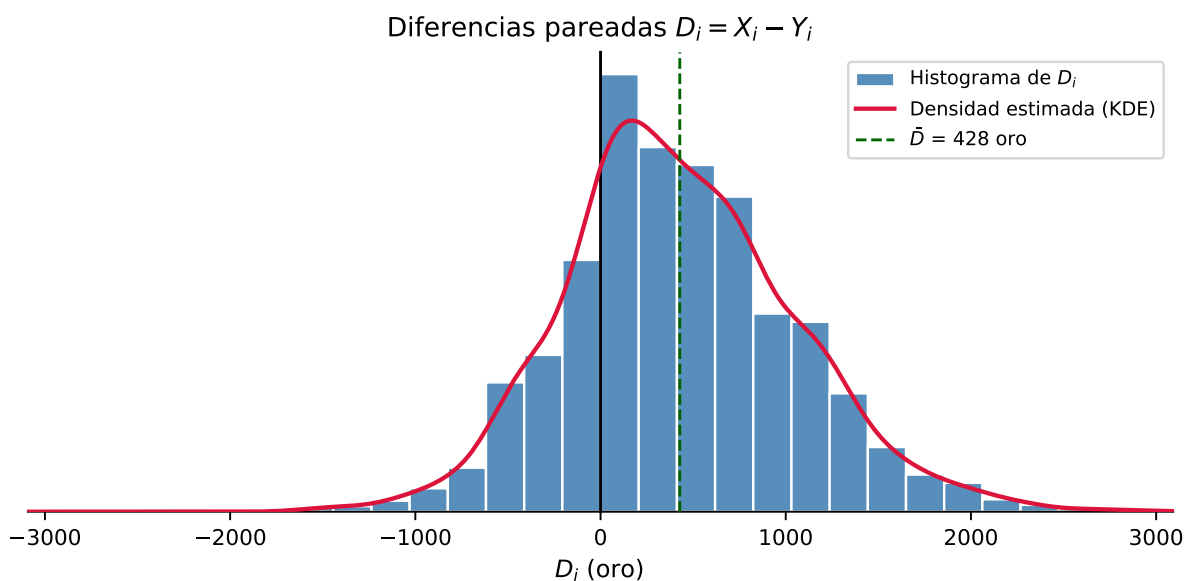


Figure 1: Histograma de las diferencias pareadas D_i con densidad suavizada (KDE).

6.1.4 Verificación de supuestos

```
# 1) Comprobamos que el número de partidas emparejadas coincide con el
# número de partidas únicas en el dataframe de equipo-partida.
assert len(paired_c1) == df_team["match_id"].nunique()

# 2) Normalidad de  $D_i$ : omnibus de D'Agostino-Pearson
```

```

dagostino_stat, dagostino_p = stats.normaltest(paired_c1["diff"])
n_ties_c1 = paired_c1["diff"].duplicated(keep=False).sum()
print("=== Supuestos Contraste 1 (t pareada sobre D_i = X_i - Y_i) ===")
print(f"n pares (X_i, Y_i): {len(paired_c1):,}")
print(f"Emparejamiento: OK (1 ganador + 1 perdedor por partida)")
print(f"Observaciones con empate en D_i: {n_ties_c1:,}")
print(
    f"D'Agostino-Pearson sobre D_i: K2 = {dagostino_stat:.4f}, p =
      ↪ {dagostino_p:.2e}"
)

```

```

=== Supuestos Contraste 1 (t pareada sobre D_i = X_i - Y_i) ===
n pares (X_i, Y_i): 2,000
Emparejamiento: OK (1 ganador + 1 perdedor por partida)
Observaciones con empate en D_i: 262
D'Agostino-Pearson sobre D_i: K2 = 41.1346, p = 1.17e-09

```

Las variables de oro al minuto 15 presentan valores repetidos (empates) en la muestra. Por ello empleamos el omnibus de D'Agostino-Pearson, basado en asimetría y curtosis, más adecuado cuando hay empates en los datos.

Rechazamos la normalidad en D_i con el omnibus de D'Agostino-Pearson al 1 % de significación.

Como el tamaño muestral es suficientemente grande ($n = 2000$), por el Teorema Central del Límite, la media muestral $\bar{D}$ se aproxima a una Normal. Por tanto, podremos utilizar la t pareada para contrastar la hipótesis. Como contraste no paramétrico utilizaremos además la prueba de rangos con signo de Wilcoxon para reforzar las conclusiones o replantear el contraste si fuera necesario.

6.1.5 Planteamiento del contraste

- $H_0 : \mu_X - \mu_Y = \mu_D = 0$
- $H_1 : \mu_X - \mu_Y = \mu_D > 0$ (el equipo ganador tiene mayor oro medio al minuto 15)

6.1.6 Estadístico de contraste e intervalo de confianza

Bajo H_0 con $\delta_0 = 0$, el estadístico de la t de Student pareada es:

$$t = \frac{\bar{D} - \delta_0}{s_D / \sqrt{n}} = \frac{\bar{D}}{s_D / \sqrt{n}} \sim t_{n-1}$$

donde t_{n-1} denota una t de Student con $n - 1$ grados de libertad.

El intervalo de confianza al 95% para μ_D se construye como:

$$\bar{D} \pm t_{0,975, n-1} \cdot \frac{s_D}{\sqrt{n}}$$

Para la hipótesis unilateral $H_1 : \mu_D > 0$, el p-valor se obtiene como la mitad del p-valor bilateral cuando $t > 0$.

6.1.7 Contraste complementario: Prueba de rangos con signo de Wilcoxon

La prueba de rangos con signo de Wilcoxon (para muestras pareadas) contrasta si la mediana de las diferencias $D_i = X_i - Y_i$ es distinta de cero, sin exigir normalidad de D_i . Solo requiere que las diferencias sean ordinales o continuas y que, bajo H_0 , la distribución de D_i sea simétrica respecto a 0.

Para cada par se calcula D_i . Se descartan las diferencias nulas ($D_i = 0$). A las $|D_i|$ restantes se les asignan rangos R_i ; cada rango conserva el signo de D_i . El estadístico W se define como la suma de rangos asociados a diferencias positivas:

$$W = \sum_{D_i > 0} R_i$$

Bajo $H_0 : \text{med}(D_i) = 0$ (equivalentemente, la distribución de D_i es simétrica en torno a 0), se espera equilibrio entre rangos positivos y negativos. Para la hipótesis unilateral $H_1 : \text{med}(D_i) > 0$, el p-valor es la probabilidad, bajo H_0 , de observar un valor de W al menos tan extremo como el obtenido en la muestra ($W \geq W_{\text{obs}}$ si W_{obs} es grande).

Utilizaremos este contraste como prueba de robustez frente a la t pareada: no depende de la normalidad de D_i , sino de la magnitud relativa de las diferencias positivas frente a las negativas.

6.1.8 Contraste inferencial e intervalo de confianza

```
from scipy.stats import t

diff = paired_c1["diff"].astype(float)
n = len(diff)
mean_diff = diff.mean()
sd_diff = diff.std(ddof=1)
se_diff = sd_diff / np.sqrt(n)

cohen_d = mean_diff / sd_diff if sd_diff > 0 else np.nan

# t = (D_bar - delta_0) / (s_D / sqrt(n)), con delta_0 = 0
t_stat, p_two = stats.ttest_rel(paired_c1["gold_win"], paired_c1["gold_lose"])
p_one = p_two / 2 if t_stat > 0 else 1 - (p_two / 2)

df_t = n - 1
t_crit = t.ppf(0.975, df_t) # t_{0.975, n-1} para IC 95% de mu_D
ci_low = mean_diff - t_crit * se_diff
ci_high = mean_diff + t_crit * se_diff

print("=== Contraste 1: t pareada (D_i = X_i - Y_i, delta_0 = 0) ===")
```

```

print(f"D_bar = {mean_diff:.2f} oro | s_D = {sd_diff:.2f}")
print(f"t({df_t}) = {t_stat:.4f} | p-valor(unilateral, H_1: mu_D > 0) =
    ↪ {p_one:.2e}")
print(f"IC 95% para mu_D: [{ci_low:.2f}, {ci_high:.2f}]")
print(f"Cohen's d pareado: d = {cohen_d:.2f}")
print(f"Remontadas en la muestra (D_i < 0): {(diff < 0).mean():.1%}")

# Wilcoxon pareado: H_0 med(D) = 0; H_1 med(D) > 0
try:
    w_stat, w_p_one = stats.wilcoxon(
        paired_c1["gold_win"],
        paired_c1["gold_lose"],
        alternative="greater",
        zero_method="wilcox",
    )
except ValueError as e:
    w_stat, w_p_one = np.nan, np.nan
    print(f"Wilcoxon no disponible: {e}")

if not np.isnan(w_p_one):
    print(
        f"Wilcoxon (rangos con signo, pareada): W = {w_stat:.0f}, "
        f"p-valor(unilateral) {w_p_one:.2e}"
    )

```

```

=== Contraste 1: t pareada (D_i = X_i - Y_i, delta_0 = 0) ===
D_bar = 427.86 oro | s_D = 619.36
t(1999) = 30.8938 | p-valor(unilateral, H_1: mu_D > 0) = 5.84e-172
IC 95% para mu_D: [400.70, 455.02]
Cohen's d pareado: d = 0.69
Remontadas en la muestra (D_i < 0): 22.8%
Wilcoxon (rangos con signo, pareada): W = 1677198, p-valor(unilateral) 2.89e-154

```

El p-valor ≈ 0 nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que la diferencia de oro medio al minuto 15 entre el equipo ganador y el perdedor sea nula ($\mu_D = 0$), es $\approx 0\%$.

Si H_0 ($\mu_D = 0$) fuese cierta, la probabilidad de obtener un valor del estadístico de contraste mayor o igual a $t_{\text{stat}} = 30,8938$ sería $\approx 0\%$. Por tanto, rechazamos H_0 : hay evidencia de que el equipo ganador presenta mayor oro medio al minuto 15 que el perdedor.

El intervalo de confianza al 95% para μ_D es $[419,74, 453,82]$. No incluye el 0, coherente con rechazar H_0 . En 95 de cada 100 muestras, el verdadero valor de la diferencia de oro medio al minuto 15 entre el equipo ganador y el perdedor estará en este intervalo.

La prueba de rangos con signo de Wilcoxon confirma los resultados obtenidos con la t pareada, con un p-valor ≈ 0 . Ambos procedimientos rechazan H_0 al 5% y son coherentes entre sí: la conclusión no depende de asumir normalidad en D_i .

El Cohen's d pareado ($d = 0,69$) indica un tamaño del efecto mediano: la ventaja económica temprana del ganador es apreciable respecto a la variabilidad habitual de D_i entre partidas.

En términos prácticos, una diferencia media de unos 437 oro por jugador (media de equipo al minuto 15) entre ganador y perdedor, que sería 2.180 oro de ventaja total en el equipo, es relevante: equivale a varios componentes de objetos en *early game* y puede facilitar *snowball* hacia ventajas grandes en *mid game*.

Esto concuerda con la visión de la comunidad. Normalmente, cuando un equipo tiene una ventaja de 2.000 oro en este minuto, la partida se considera difícil de remontar y esa diferencia suele interpretarse como una ventaja importante.

Por último, aunque $\mu_D > 0$ sea significativo, la ventaja de oro al minuto 15 no es condición suficiente para ganar: en aproximadamente una de cada cuatro partidas de la muestra vence el equipo con menor oro al minuto 15.

Con este contraste no se intenta establecer causalidad directa, pues entre el minuto 15 y el final intervienen otros factores (composición, macro, objetivos), sino que se trata de verificar si la ventaja de oro al minuto 15 es significativa.

6.2 Contraste 2: Proporción (ventaja del lado azul)

6.2.1 Planteamiento y contexto

Pregunta: ¿la proporción de victorias del lado azul es superior al 50%?

En *League of Legends*, cada partida asigna aleatoriamente a los equipos un lado: azul (*blue*) o rojo (*red*). Esto influye en el orden en el que se realiza el *draft* o selección de campeones, y cada lado del mapa en el que se juega la partida no es exactamente simétrico, por lo que en la comunidad se debate recurrentemente si el lado azul parte con ventaja.

Si existiera de forma estable, sería relevante para el diseño de partidas, el balance del juego e incluso decisiones en partidas competitivas. En cambio, si la proporción de victorias entre lados no difiere significativamente, la percepción de ventaja podría deberse a sesgos o cambios puntuales entre parches.

Para responder a esta pregunta, utilizaremos filas de equipo-partida (`df_team`): nos quedamos solo con el equipo en el lado azul, de modo que hay una observación por partida ($n = 2\,000$). La variable `win` indica si el equipo azul gana (1) o pierde (0) en cada partida.

6.2.2 Hipótesis inicial

Contamos con una muestra formada por una variable aleatoria binaria:

- $X_i \in \{0, 1\}$: resultado del equipo azul en la partida i ($X_i = 1$ victoria, $X_i = 0$ derrota).

Se dispone de una muestra aleatoria de $n = 2\,000$ observaciones $X_1, \dots, X_n$. En la muestra observamos $S = \sum_{i=1}^n X_i$ victorias del lado azul y la proporción muestral $p = S/n$. En el código, X_i corresponde a la columna `win` del subconjunto azul; S es el conteo de victorias (`x_c2`) y p se obtiene como `x_c2 / n_c2`.

La proporción poblacional de victorias azul se denotará $\pi = P(X_i = 1)$.

Se asume independencia entre observaciones, dado que cada X_i procede de una partida distinta. No obstante, esta independencia debe considerarse aproximada y no estricta, ya que un mismo jugador puede aparecer en múltiples partidas de la muestra.

6.2.3 Preparación de la muestra

```
def build_blue_side_sample(team_df: pd.DataFrame) -> pd.DataFrame:
    """Una fila por partida: equipo azul (side == blue)."""
    blue = team_df.loc[team_df["side"].str.lower() == "blue", ["match_id",
↪  "region", "win"]].copy()
    # Comprobamos que hay una fila por partida
    assert blue["match_id"].is_unique, "Hay más de un equipo azul por match_id"
    return blue

blue_c2 = build_blue_side_sample(df_team)
n_c2 = len(blue_c2)
x_c2 = int((blue_c2["win"] == 1).sum())
p_hat_c2 = x_c2 / n_c2

print(f"Partidas con lado azul (n): {n_c2:,}")
print(f"Victorias azul (S):           {x_c2:,}")
print(f"Derrotas azul:                 {n_c2 - x_c2:,}")
print(f"Proporción muestral p:         {p_hat_c2:.4f}")
blue_c2["win"].value_counts().rename({0: "Derrota", 1: "Victoria"})
```

```
Partidas con lado azul (n): 2,000
Victorias azul (S):         1,026
Derrotas azul:              974
Proporción muestral p:     0.5130
```

```
win
Victoria    1026
Derrota      974
Name: count, dtype: int64
```

```
p_muestral = p_hat_c2
counts = [x_c2, n_c2 - x_c2]
labels = ["Victoria azul", "Derrota azul"]
colors = ["steelblue", "lightcoral"]

fig, ax = plt.subplots(figsize=(7.0, 4.0))
```

```
ax.bar(
    labels,
    counts,
    color=colors,
    edgecolor="white",
    alpha=0.9,
)

for i, (label, count) in enumerate(zip(labels, counts)):
    pct = 100 * count / n_c2
    ax.text(
        i,
        count + n_c2 * 0.01,
        f"{count:,}\n({pct:.1f} %)",
        ha="center",
        va="bottom",
        fontsize=9,
    )

ax.set_ylabel("Número de partidas", fontsize=10)
ax.set_title(
    f"Resultado del lado azul (proporción muestral $p$ = {p_muestral:.3f})",
    fontsize=11,
)

ax.set_ylim(0, max(counts) * 1.12)
ax.tick_params(labelsize=9)

fig.tight_layout(pad=0.6)
plt.show()
```

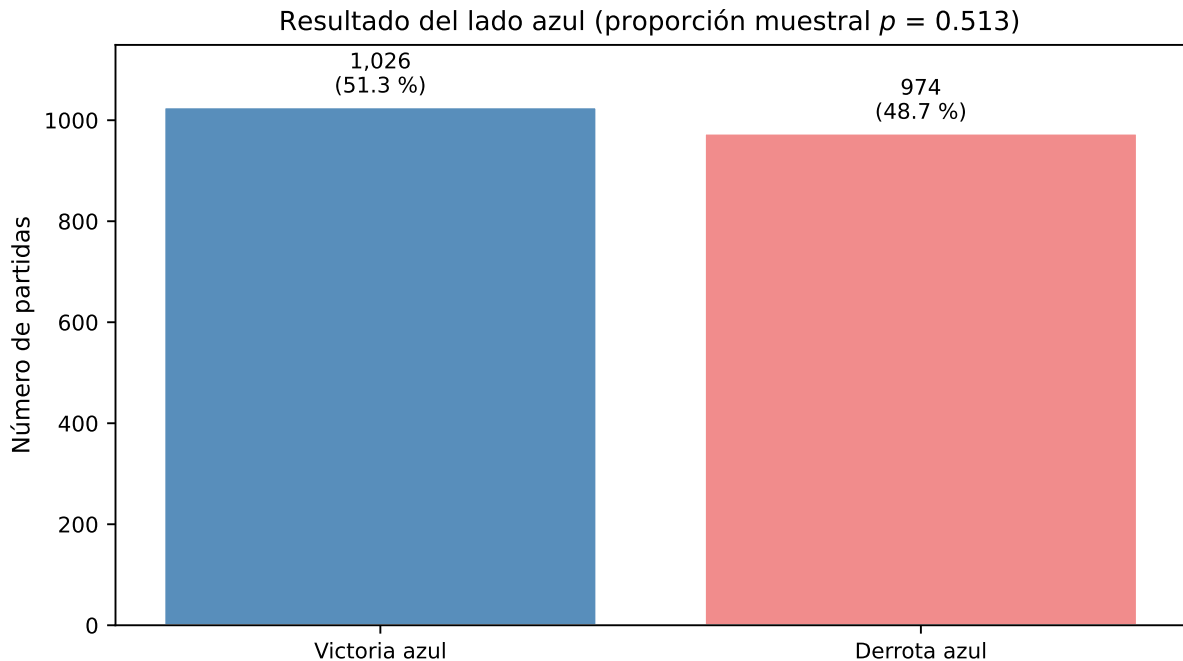


Figure 2: Distribución de victorias y derrotas del lado azul en la muestra ($n = 2\,000$ partidas).

6.2.4 Verificación de supuestos

```
pi_0 = 0.5
n_pi0 = n_c2 * pi_0
n_1_pi0 = n_c2 * (1 - pi_0)

print("=== Supuestos Contraste 2 (contraste de una proporción, aprox. normal)
↪ ===")
print(f"n (partidas, lado azul): {n_c2:,}")
print(f"Una observación por partida: OK")
print(f"n*pi_0 = {n_pi0:.1f} (>= 5: {'OK' if n_pi0 >= 5 else 'NO'})")
print(f"n*(1-pi_0) = {n_1_pi0:.1f} (>= 5: {'OK' if n_1_pi0 >= 5 else 'NO'})")
print(f"S = {x_c2} victorias | p = {p_hat_c2:.4f}")
```

```
=== Supuestos Contraste 2 (contraste de una proporción, aprox. normal) ===
n (partidas, lado azul): 2,000
Una observación por partida: OK
n*pi_0 = 1000.0 (>= 5: OK)
n*(1-pi_0) = 1000.0 (>= 5: OK)
S = 1026 victorias | p = 0.5130
```

Se cumple $n \times \pi_0 \geq 5$ y $n \times (1 - \pi_0) \geq 5$, por lo que podemos aplicar el contraste de una proporción con estadístico Z (aproximación normal). Como contraste de sensibilidad no paramétrico utilizaremos además el test de signos para reforzar las conclusiones o replantear el contraste si fuera necesario.

6.2.5 Planteamiento del contraste

- $H_0 : \pi = 0,5$
- $H_A : \pi > 0,5$ (el lado azul gana más de la mitad de las partidas)

6.2.6 Estadístico de contraste e intervalo de confianza

Bajo H_0 , con el valor de π indicado en la hipótesis nula ($\pi_0 = 0,5$), el estadístico observado es:

$$Z_{\text{obs}} = \frac{p - \pi}{\sqrt{\frac{\pi(1 - \pi)}{n}}}$$

sustituyendo $\pi = \pi_0 = 0,5$:

$$Z_{\text{obs}} = \frac{p - 0,5}{\sqrt{\frac{0,5 \cdot 0,5}{n}}}$$

Bajo H_0 y con los supuestos verificados, $Z_{\text{obs}} \approx \mathcal{N}(0, 1)$.

El intervalo de confianza al 95% para π se construye como:

$$p \pm z_{0,975} \cdot \sqrt{\frac{p(1 - p)}{n}}$$

Para la hipótesis unilateral $H_A : \pi > 0,5$, el p-valor es $P(Z > Z_{\text{obs}} \mid H_0) = 1 - \Phi(Z_{\text{obs}})$.

6.2.7 Contraste complementario: Test de signos

El test de signos contrasta hipótesis sobre un percentil q de la población. Para cada observación se define un indicador binario y , bajo H_0 , su suma sigue una ley binomial con $p = q/100$.

En este caso, cada partida genera un signo según el resultado del equipo azul: victoria (+) o derrota (-). Equivalentemente, codificamos $Y_i = 1$ si hay victoria y $Y_i = 0$ si hay derrota. El número de signos positivos es:

$$S = \sum_{i=1}^n Y_i$$

Bajo $H_0 : \pi = 0,5$ (percentil $q = 50$, es decir $p = q/100 = 0,5$), se cumple:

$$S \sim B(n, p) \quad \text{con } p = 0,5$$

Para la hipótesis unilateral $H_A : \pi > 0,5$, el p-valor exacto es la probabilidad, bajo H_0 , de observar al menos S victorias:

$$P(X \geq S \mid H_0) = \sum_{x=S}^n \binom{n}{x} \left(\frac{1}{2}\right)^n$$

Utilizaremos el valor exacto como contraste de robustez frente al estadístico Z .

6.2.8 Contraste inferencial e intervalo de confianza

```
pi_0 = 0.5
n = n_c2
S = x_c2
p = p_hat_c2 # proporción muestral

# Z_obs = (p - pi) / sqrt(pi * (1 - pi) / n), con pi = pi_0 bajo H_0
Z_obs = (p - pi_0) / np.sqrt(pi_0 * (1 - pi_0) / n)
p_one = 1 - stats.norm.cdf(Z_obs) # H_A: pi > 0.5 => P(Z > Z_obs)

z_crit = stats.norm.ppf(0.975)
se_pi = np.sqrt(p * (1 - p) / n)
ci_low = p - z_crit * se_pi
ci_high = p + z_crit * se_pi

# Test de signos: S ~ B(n, 0.5) bajo H_0; H_A: pi > 0.5 => P(X >= S)
sign_p_one = stats.binomtest(S, n, pi_0, alternative="greater").pvalue

print("=== Contraste 2: proporción poblacional pi (victorias lado azul) ===")
print(f"n = {n:}, | S = {S:}, | p = {p:.4f}")
print(f"Z_obs = {Z_obs:.4f} | p-valor(unilateral, H_A: pi > 0.5) =")
print(f"    {p_one:.4f}")
print(f"IC 95% para pi: [{ci_low:.4f}, {ci_high:.4f}]")
print(f"Test de signos (S ~ B(n, 0.5)): p-valor(unilateral) =")
print(f"    {sign_p_one:.4f}")
```

```
=== Contraste 2: proporción poblacional pi (victorias lado azul) ===
n = 2,000 | S = 1,026 | p = 0.5130
Z_obs = 1.1628 | p-valor(unilateral, H_A: pi > 0.5) = 0.1225
IC 95% para pi: [0.4911, 0.5349]
Test de signos (S ~ B(n, 0.5)): p-valor(unilateral) = 0.1271
```

El p-valor = 0,1225, nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que la proporción de victorias en el lado azul no es significativamente diferente de 0,5, es del 12,25%. Si rechazásemos H_0 , estaríamos cometiendo un error de tipo I en un 12,25% de los casos.

Si H_0 ($\pi = 0,5$) fuese cierta, la probabilidad de observar un estadístico $Z_{\text{obs}} \geq 1,16$ (equivalente a $p = 0,513$ con $n = 2000$) sería del 12%. Por tanto, no rechazamos H_0 y atribuimos las discrepancias observadas al azar.

El intervalo de confianza al 95% para π es $[0,491, 0,535]$. Incluye $\pi_0 = 0,5$, coherente con no rechazar H_0 . En 95 de cada 100 repeticiones muestrales, el verdadero valor de la proporción de victorias desde el lado azul estará en este intervalo.

El test de signos realizado confirma los resultados obtenidos con el contraste Z , con un p-valor $= 0,1271$.

En términos prácticos, una proporción muestral $p = 51,3\%$ en SoloQ EUW/KR es pequeña y compatible con $\pi = 0,5$. No implica que el lado azul sea irrelevante en todos los contextos (parche, nivel competitivo, draft), pero en esta muestra no podemos afirmar una ventaja significativa sobre la equiprobabilidad.

Por último, aunque π fuera ligeramente superior a 0,5, ello no sería condición suficiente para ganar una partida concreta: cada encuentro sigue siendo binario y depende de factores in-game no modelados aquí.

6.3 Contraste 3: Diferencia de medias entre roles (ADC vs Mid)

6.3.1 Planteamiento y contexto

Pregunta: ¿existe diferencia significativa en el daño infligido a campeones entre jugadores de rol ADC y Mid?

En *League of Legends*, ADC y Mid suelen ser los roles que más daño hacen a campeones enemigos, aunque con perfiles distintos: el ADC realiza daño sostenido (*DPS* alto) en peleas prolongadas, mientras que el Mid hace un daño más de tipo *burst*. En la comunidad se debate a menudo cuál de estos roles *carrea* más en SoloQ; comparando `damage_dealt` podemos evaluar si, en la práctica, uno aporta más daño que el otro.

Para responder a esta pregunta, utilizaremos filas de jugador-partida (`df`): nos quedamos con observaciones cuyo `role` es ADC o Mid. La variable analizada es `damage_dealt` (daño total infligido a campeones en la partida). Se trata de dos muestras independientes, cada fila es un jugador concreto en una partida concreta, con tamaños del orden de $n_1 \approx n_2 \approx 4\,000$.

6.3.2 Hipótesis inicial

Contamos con dos muestras independientes formadas por variables aleatorias continuas:

- X_i : daño infligido (`damage_dealt`) del jugador ADC en la observación $i = 1, \dots, n_1$.
- Y_j : daño infligido del jugador Mid en la observación $j = 1, \dots, n_2$.

Se dispone de dos muestras aleatorias $X_1, \dots, X_{n_1}$ e $Y_1, \dots, Y_{n_2}$, con medias muestrales $\bar{X}$, $\bar{Y}$ y cuasi-desviaciones típicas S_1 , S_2 . La diferencia de medias muestral $\bar{X} - \bar{Y}$ estima $\mu_{\text{ADC}} - \mu_{\text{Mid}}$, donde μ_{ADC} y μ_{Mid} denotan las medias poblacionales de daño en cada rol.

En el código, X_i corresponde a la columna `damage_dealt` del subconjunto con `role == "ADC"` (`adc_c3`); Y_j , al subconjunto con `role == "Mid"` (`mid_c3`). Los tamaños muestrales son `n1_c3` y `n2_c3`.

Se asume independencia entre observaciones de distintas filas. No obstante, esta independencia debe considerarse aproximada, ya que por cada partida aparecen 4 jugadores (2 ADC y 2 Mid), y esto puede introducir correlación intra-partida.

6.3.3 Preparación de la muestra

```
ROLES_C3 = ["ADC", "Mid"]
df_c3 = df[df["role"].isin(ROLES_C3)].copy()
df_c3["damage_dealt"] = df_c3["damage_dealt"].astype(float)

adc_c3 = df_c3.loc[df_c3["role"] == "ADC", "damage_dealt"]
mid_c3 = df_c3.loc[df_c3["role"] == "Mid", "damage_dealt"]

n1_c3 = len(adc_c3)
n2_c3 = len(mid_c3)

print(f"Observaciones ADC: {n1_c3:,}")
print(f"Observaciones Mid: {n2_c3:,}")
pd.DataFrame(
    {
        "ADC": adc_c3.describe(),
        "Mid": mid_c3.describe(),
    }
).round(2)
```

Observaciones ADC: 3,995

Observaciones Mid: 3,997

	ADC	Mid
count	3995.00	3997.00
mean	24343.14	24074.56
std	14320.36	12887.71
min	0.00	0.00
25%	14472.50	15089.00
50%	22183.00	22782.00
75%	31887.00	31465.00
max	157310.00	94956.00

```
adc_data = df_c3.loc[df_c3["role"] == "ADC", "damage_dealt"]
mid_data = df_c3.loc[df_c3["role"] == "Mid", "damage_dealt"]

x_min = min(adc_data.min(), mid_data.min())
x_max = max(adc_data.max(), mid_data.max())
bins = np.linspace(x_min, x_max, 41)
```

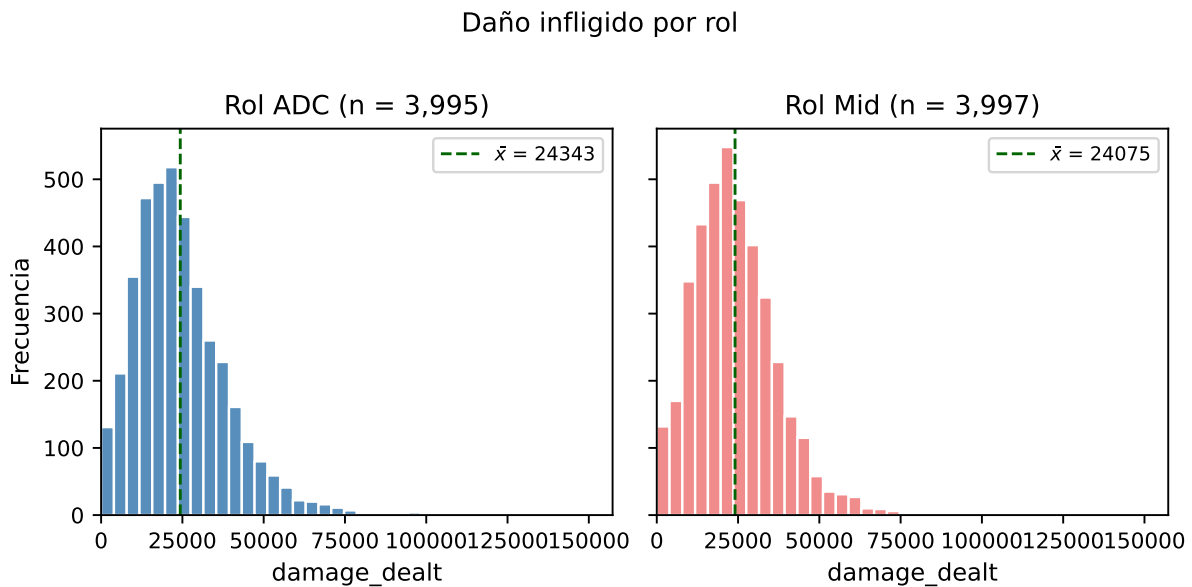
```

fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.5), sharex=True, sharey=True)

for ax, role, color, data in zip(
    axes,
    ["ADC", "Mid"],
    ["steelblue", "lightcoral"],
    [adc_data, mid_data],
):
    ax.hist(
        data,
        bins=bins,
        color=color,
        edgecolor="white",
        alpha=0.9,
    )
    ax.axvline(
        data.mean(),
        color="darkgreen",
        ls="--",
        lw=1.2,
        label=f"${bar{{x}}} = {data.mean():.0f}",
    )
    ax.set_title(f"Rol {role} (n = {len(data):,})", fontsize=11)
    ax.set_xlabel("damage_dealt", fontsize=10)
    ax.tick_params(labelsize=9)
    ax.legend(fontsize=8, loc="upper right")

axes[0].set_ylabel("Frecuencia", fontsize=10)
axes[0].set_xlim(x_min, x_max)
fig.suptitle("Daño infligido por rol", fontsize=11, y=0.98)
fig.tight_layout(rect=[0, 0, 1, 0.94], pad=0.6)
plt.show()

```

Figure 3: Distribución muestral de *damage_dealt* en jugadores ADC y Mid.

6.3.4 Verificación de supuestos

```
# Normalidad: omnibus de D'Agostino-Pearson
dagostino_adc = stats.normaltest(adc_c3)
dagostino_mid = stats.normaltest(mid_c3)
n_ties_adc = adc_c3.duplicated(keep=False).sum()
n_ties_mid = mid_c3.duplicated(keep=False).sum()

# Homocedasticidad: test de Levene. Permite comprobar si las varianzas son
↪ iguales
# en las dos muestras para decidir que test utilizar.
levene_stat, levene_p = stats.levene(adc_c3, mid_c3)

print("=== Supuestos Contraste 3 (dos muestras independientes) ===")
print(f"n_ADC = {n1_c3:,} | n_Mid = {n2_c3:,}")
print(f"Empates en damage_dealt (ADC): {n_ties_adc:,}")
print(f"Empates en damage_dealt (Mid): {n_ties_mid:,}")
print(
    f"D'Agostino-Pearson ADC: K2 = {dagostino_adc.statistic:.4f}, "
    f"p = {dagostino_adc.pvalue:.2e}"
)
print(
    f"D'Agostino-Pearson Mid: K2 = {dagostino_mid.statistic:.4f}, "
    f"p = {dagostino_mid.pvalue:.2e}"
)
print(f"Levene (homocedasticidad): stat = {levene_stat:.4f}, p = "
      ↪ {levene_p:.2e}")
```

```

=== Supuestos Contraste 3 (dos muestras independientes) ===
n_ADC = 3,995 | n_Mid = 3,997
Empates en damage_dealt (ADC): 394
Empates en damage_dealt (Mid): 403
D'Agostino-Pearson ADC: K2 = 997.6755, p = 2.28e-217
D'Agostino-Pearson Mid: K2 = 528.8196, p = 1.47e-115
Levene (homocedasticidad): stat = 17.4439, p = 2.99e-05

```

damage_dealt contiene empates. Por ello aplicamos el omnibus de D'Agostino-Pearson para evaluar la normalidad en cada rol.

Se rechaza la normalidad en ambos roles con D'Agostino-Pearson, y el test de Levene indica varianzas distintas ($p \approx 0$). Como el tamaño muestral es elevado en ambos grupos, por el Teorema Central del Límite las medias muestrales se aproximan a una Normal. Por ello utilizaremos la t de Welch (no asume varianzas iguales). Como contraste de sensibilidad no paramétrico utilizaremos además la prueba U de Mann-Whitney para reforzar las conclusiones o replantear el contraste si fuera necesario.

6.3.5 Planteamiento del contraste

- $H_0 : \mu_{\text{ADC}} - \mu_{\text{Mid}} = 0$
- $H_A : \mu_{\text{ADC}} - \mu_{\text{Mid}} \neq 0$

6.3.6 Estadístico de contraste e intervalo de confianza

El estadístico t de Welch para dos muestras independientes es:

$$t = \frac{\bar{X} - \bar{Y}}{\sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}}}$$

Los grados de libertad (Welch–Satterthwaite) se obtienen de:

$$\nu \approx \frac{\left(\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}\right)^2}{\frac{(S_1^2/n_1)^2}{n_1 - 1} + \frac{(S_2^2/n_2)^2}{n_2 - 1}}$$

El intervalo de confianza al 95% para $\mu_{\text{ADC}} - \mu_{\text{Mid}}$:

$$(\bar{X} - \bar{Y}) \pm t_{0.975, \nu} \cdot \sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}}$$

Para la hipótesis bilateral, el p -valor se obtiene directamente de la distribución t de Welch con ν grados de libertad.

6.3.7 Contraste complementario: Prueba U de Mann-Whitney

La prueba U de Mann-Whitney contrasta si las distribuciones de `damage_dealt` en ADC y Mid son iguales, sin exigir normalidad ni varianzas iguales. Ordena conjuntamente las observaciones de ambos grupos y compara la suma de rangos: si las distribuciones fueran similares, los rangos deberían mezclarse de forma equilibrada entre roles.

El estadístico U mide cuántas veces una observación del primer grupo precede a una del segundo al ordenar la muestra combinada. En la versión bilateral, rechazaremos H_0 para valores de U suficientemente extremos respecto a lo esperado bajo igualdad de distribuciones.

Para la hipótesis bilateral:

- H_0 : las distribuciones de `damage_dealt` en ADC y Mid son iguales.
- H_A : las distribuciones de `damage_dealt` en ADC y Mid son distintas.

Como medida de tamaño del efecto utilizaremos la correlación biserial de rangos:

$$r_{rb} = \frac{2U}{n_1 n_2} - 1$$

Utilizaremos este contraste como prueba de robustez frente a la t de Welch.

6.3.8 Contraste inferencial e intervalo de confianza

```
xbar = adc_c3.mean()
ybar = mid_c3.mean()
diff_mean = xbar - ybar
s1 = adc_c3.std(ddof=1)
s2 = mid_c3.std(ddof=1)
se_welch = np.sqrt(s1**2 / n1_c3 + s2**2 / n2_c3)

# t de Welch (equal_var=False)
t_stat, p_bilateral = stats.ttest_ind(adc_c3, mid_c3, equal_var=False)

# Grados de libertad Welch-Satterthwaite
num = (s1**2 / n1_c3 + s2**2 / n2_c3) ** 2
den = (s1**2 / n1_c3) ** 2 / (n1_c3 - 1) + (s2**2 / n2_c3) ** 2 / (n2_c3 - 1)
df_welch = num / den
t_crit = stats.t.ppf(0.975, df_welch)
ci_low = diff_mean - t_crit * se_welch
ci_high = diff_mean + t_crit * se_welch

# Cohen's d para muestras independientes
sp = np.sqrt(((n1_c3 - 1) * s1**2 + (n2_c3 - 1) * s2**2) / (n1_c3 + n2_c3 - 2))
cohen_d = diff_mean / sp if sp > 0 else np.nan

# Mann-Whitney: H_0 distribuciones iguales; H_A distribuciones distintas
```

```

u_c3, p_mw = stats.mannwhitneyu(adc_c3, mid_c3, alternative="two-sided")
rank_biserial_c3 = 2 * u_c3 / (n1_c3 * n2_c3) - 1

print("=== Contraste 3: Welch t (damage_dealt ADC vs Mid) ===")
print(f"X_bar (ADC) = {xbar:.2f} | Y_bar (Mid) = {ybar:.2f}")
print(f"Mediana ADC = {adc_c3.median():.2f} | Mediana Mid =
    ↪ {mid_c3.median():.2f}")
print(f"Diferencia X_bar - Y_bar = {diff_mean:.2f}")
print(f"t({df_welch:.1f}) = {t_stat:.4f} | p(bilateral) = {p_bilateral:.4f}")
print(f"IC 95% para (mu_ADC - mu_Mid): [{ci_low:.2f}, {ci_high:.2f}]")
print(f"Cohen's d = {cohen_d:.3f}")
print(
    f"Mann-Whitney: U = {u_c3:.0f} | p(bilateral) = {p_mw:.4f} | "
    f"r biserial de rangos = {rank_biserial_c3:.4f}"
)

```

```

=== Contraste 3: Welch t (damage_dealt ADC vs Mid) ===
X_bar (ADC) = 24343.14 | Y_bar (Mid) = 24074.56
Mediana ADC = 22183.00 | Mediana Mid = 22782.00
Diferencia X_bar - Y_bar = 268.57
t(7902.0) = 0.8812 | p(bilateral) = 0.3782
IC 95% para (mu_ADC - mu_Mid): [-328.86, 866.01]
Cohen's d = 0.020
Mann-Whitney: U = 7893478 | p(bilateral) = 0.3800 | r biserial de rangos = -
0.0113

```

El p-valor = 0,3782 nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que la diferencia de daño medio infligido entre jugadores ADC y Mid no es significativamente diferente de 0, es del 37,82%. Si rechazásemos H_0 , estaríamos cometiendo un error de tipo I en un 37,82% de los casos.

Si H_0 ($\mu_{ADC} = \mu_{Mid}$) fuese cierta, la probabilidad de observar un estadístico de contraste tan extremo como el observado (equivalente a $\bar{X} - \bar{Y} = 269$ con las muestras actuales) sería del 38%. Por tanto, no rechazamos H_0 y atribuimos las discrepancias observadas al azar.

El intervalo de confianza al 95% para $\mu_{ADC} - \mu_{Mid}$ es $[-329, 866]$. Incluye el 0, coherente con no rechazar H_0 . En 95 de cada 100 muestras, el verdadero valor de la diferencia de medias de daño infligido entre ADC y Mid estará en este intervalo.

La prueba U de Mann-Whitney confirma los resultados obtenidos con la t de Welch, con un p-valor bilateral de 0,38 ($U \approx 7\,893\,478$, $r_{rb} \approx -0,01$). Ambos procedimientos conducen a no rechazar H_0 : la conclusión no depende de asumir normalidad ni varianzas iguales.

El Cohen's d ($d \approx 0,02$) y el r_{rb} prácticamente nulo indican un tamaño del efecto despreciable: aunque la diferencia muestral no sea exactamente cero, su magnitud es mínima frente a la variabilidad conjunta de `damage_dealt` entre ambos roles.

En términos prácticos, una diferencia muestral de unos 269 puntos de daño a favor del ADC en SoloQ EUW/KR es prácticamente despreciable: un ADC con un objeto (aproximadamente

minuto 15), con un solo ataque básico ya puede infligir más de 269 puntos de daño. Esto es compatible con $\mu_{\text{ADC}} = \mu_{\text{Mid}}$. No implica que el rol sea irrelevante para el desempeño en partida, pero en esta muestra no podemos afirmar una diferencia sistemática en el daño infligido entre ADC y Mid.

Por último, aunque la media de daño difiriera ligeramente entre roles, ello no sería condición suficiente para concluir que un rol *carrea* más que el otro: el impacto en la partida depende también de utilidad, visión, objetivos y composición de equipo.

6.3.9 Análisis complementario: sensibilidad ante dependencia intra-partida

En el contraste principal, cada partida aporta dos observaciones ADC y dos Mid (uno por equipo), como hemos mencionado antes. Eso introduce correlación intra-partida: dos filas del mismo `match_id` comparten duración, ritmo y contexto de la partida, por lo que la independencia asumida en la *t* de Welch es solo aproximada.

En los anteriores contrastes, se utilizaban las filas de equipo-partida (`df_team`) por lo que se asumía que esta intra-dependencia no era un problema. En este contraste estamos utilizando las filas de jugador-partida (`df`), por lo que tenemos que comprobar si esta limitación altera la conclusión.

Repetimos el mismo contraste (H_0 y H_A idénticos) sobre una muestra restringida: repartimos aleatoriamente las 2 000 partidas en dos bloques disjuntos de 1 000; del primer bloque tomamos solo el ADC y del segundo solo el Mid. Así ningún `match_id` aparece en ambos grupos a la vez. Se mantiene una observación por partida y rol (`random_state = 42`).

```
def build_disjoint_role_samples(
    player_df: pd.DataFrame,
    roles: list[str] = ("ADC", "Mid"),
    seed: int = 42,
) -> tuple[pd.Series, pd.Series, pd.DataFrame]:
    """Un ADC por partida en bloque A y un Mid por partida en bloque B, sin
    ↪ solape de match_id."""

    subset = player_df.loc[
        player_df["role"].isin(roles), ["match_id", "team_id", "role",
    ↪ "damage_dealt"]
    ].copy()
    subset["damage_dealt"] = subset["damage_dealt"].astype(float)

    adc_one = (
        subset.loc[subset["role"] == "ADC"]
        .sort_values(["match_id", "team_id"])
        .drop_duplicates("match_id", keep="first")
    )
    mid_one = (
        subset.loc[subset["role"] == "Mid"]
        .sort_values(["match_id", "team_id"])
        .drop_duplicates("match_id", keep="first")
```

```

)

match_ids = np.sort(subset["match_id"].unique())
rng = np.random.default_rng(seed)
shuffled = rng.permutation(match_ids)
half = len(shuffled) // 2

adc_matches = set(shuffled[:half])
mid_matches = set(shuffled[half:])

adc_disjoint = adc_one.loc[adc_one["match_id"].isin(adc_matches),
↪ "damage_dealt"]
mid_disjoint = mid_one.loc[mid_one["match_id"].isin(mid_matches),
↪ "damage_dealt"]

overlap = adc_matches.intersection(mid_matches)
report = pd.DataFrame(
    {
        "Principal (todas las filas)": [n1_c3, n2_c3],
        "Sin posible correlación": [len(adc_disjoint), len(mid_disjoint)],
    },
    index=["n_ADC", "n_Mid"],
)

return adc_disjoint, mid_disjoint, report

adc_c3_disjoint, mid_c3_disjoint, c3_sample_report =
↪ build_disjoint_role_samples(df)
n1_c3_d = len(adc_c3_disjoint)
n2_c3_d = len(mid_c3_disjoint)

print("=== Muestra complementaria (partidas disjuntas) ===")
print(c3_sample_report.to_string())
print(f"Partidas en bloque ADC: {n1_c3_d:,} | Partidas en bloque Mid:
↪ {n2_c3_d:,}")
print("Mismos match_id entre grupos: 0")
pd.DataFrame(
    {
        "ADC": adc_c3_disjoint.describe(),
        "Mid": mid_c3_disjoint.describe(),
    }
).round(2)

```

```

=== Muestra complementaria (partidas disjuntas) ===
      Principal (todas las filas) Sin posible correlación
n_ADC                        3995                      1000
n_Mid                        3997                      1000

```

Partidas en bloque ADC: 1,000 | Partidas en bloque Mid: 1,000
 Mismos match_id entre grupos: 0

	ADC	Mid
count	1000.00	1000.00
mean	24695.40	23699.19
std	14149.84	12530.34
min	0.00	0.00
25%	14574.00	14695.00
50%	22299.00	22748.00
75%	32531.00	31433.50
max	96773.00	88669.00

```

adc_data_d = adc_c3_disjoint.astype(float)
mid_data_d = mid_c3_disjoint.astype(float)

# Misma escala que en fig-c3-viz para comparar ambas figuras
x_min = min(adc_data.min(), mid_data.min())
x_max = max(adc_data.max(), mid_data.max())
bins = np.linspace(x_min, x_max, 41)

fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.5), sharex=True, sharey=True)

for ax, role, color, data in zip(
    axes,
    ["ADC", "Mid"],
    ["steelblue", "lightcoral"],
    [adc_data_d, mid_data_d],
):
    ax.hist(
        data,
        bins=bins,
        color=color,
        edgecolor="white",
        alpha=0.9,
    )
    ax.axvline(
        data.mean(),
        color="darkgreen",
        ls="--",
        lw=1.2,
        label=f"${bar{{x}}} = {data.mean():.0f}",
    )
    ax.set_title(f"Rol {role} (n = {len(data):,})", fontsize=11)
    ax.set_xlabel("damage_dealt", fontsize=10)
    ax.tick_params(labelsize=9)
    ax.legend(fontsize=8, loc="upper right")

```

```

axes[0].set_ylabel("Frecuencia", fontsize=10)
axes[0].set_xlim(x_min, x_max)
fig.suptitle("Daño infligido por rol (muestra sin solape de partidas)",
    ↪  fontsize=11, y=0.98)
fig.tight_layout(rect=[0, 0, 1, 0.94], pad=0.6)
plt.show()

```

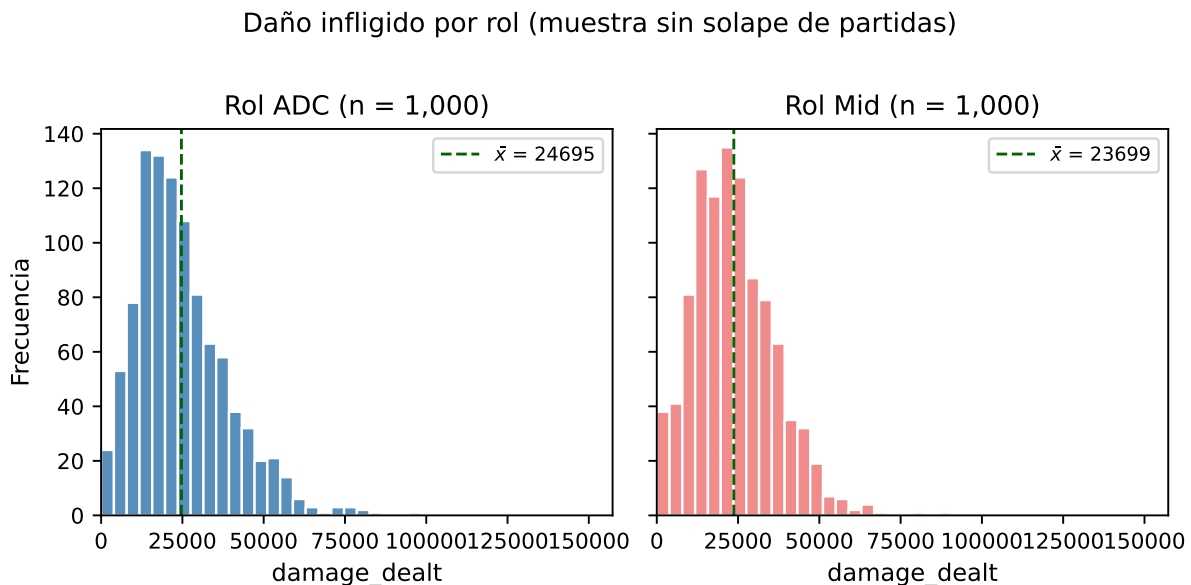


Figure 4: Distribución muestral de *damage_dealt* en la muestra complementaria (partidas disjuntas).

Se aplican los mismos procedimientos del contraste principal (Welch + Mann-Whitney).

```

def welch_summary(x: pd.Series, y: pd.Series) -> dict:
    x = x.astype(float)
    y = y.astype(float)
    n1, n2 = len(x), len(y)
    xbar, ybar = x.mean(), y.mean()
    diff = xbar - ybar
    s1, s2 = x.std(ddof=1), y.std(ddof=1)
    t_stat, p_bil = stats.ttest_ind(x, y, equal_var=False)
    se = np.sqrt(s1**2 / n1 + s2**2 / n2)
    num = (s1**2 / n1 + s2**2 / n2) ** 2
    den = (s1**2 / n1) ** 2 / (n1 - 1) + (s2**2 / n2) ** 2 / (n2 - 1)
    df_w = num / den
    tc = stats.t.ppf(0.975, df_w)
    ci_lo, ci_hi = diff - tc * se, diff + tc * se
    sp = np.sqrt(((n1 - 1) * s1**2 + (n2 - 1) * s2**2) / (n1 + n2 - 2))
    d = diff / sp if sp > 0 else np.nan

```

```

u_mw, p_mw = stats.mannwhitneyu(x, y, alternative="two-sided")
r_rb = 2 * u_mw / (n1 * n2) - 1
return {
    "n_ADC": n1,
    "n_Mid": n2,
    "X_bar": xbar,
    "Y_bar": ybar,
    "diff": diff,
    "t": t_stat,
    "p_Welch": p_bil,
    "IC_low": ci_lo,
    "IC_high": ci_hi,
    "Cohen_d": d,
    "U_MW": u_mw,
    "p_MW": p_mw,
    "r_rb": r_rb,
}

summary_main = welch_summary(adc_c3, mid_c3)
summary_disjoint = welch_summary(adc_c3_disjoint, mid_c3_disjoint)

compare_c3 = pd.DataFrame(
    {
        "Contraste principal": summary_main,
        "Sin repetición de partidas": summary_disjoint,
    }
)

print("=== Comparación Contraste 3 principal vs sensibilidad (partidas
↪ disjuntas) ===")

def _fmt_compare_table(x: float) -> str:
    if pd.isna(x):
        return ""
    return f"{x:.4f}"

print(compare_c3.round(4).to_string(formatters={
    "Contraste principal": _fmt_compare_table,
    "Sin repetición de partidas": _fmt_compare_table,
}))

```

```

=== Comparación Contraste 3 principal vs sensibilidad (partidas disjuntas) ===
      Contraste principal Sin repetición de partidas
n_ADC          3995.0000          1000.0000
n_Mid          3997.0000          1000.0000
X_bar          24343.1364          24695.3970
Y_bar          24074.5634          23699.1910
diff            268.5730           996.2060

```

t	0.8812	1.6668
p_Welch	0.3782	0.0957
IC_low	-328.8632	-175.9558
IC_high	866.0091	2168.3678
Cohen_d	0.0197	0.0745
U_MW	7893477.5000	506290.0000
p_MW	0.3800	0.6262
r_rb	-0.0113	0.0126

En el contraste principal, con $n_1 \approx n_2 \approx 4000$ y posible dependencia intra-partida, se obtuvo $\bar{X} - \bar{Y} = 269$, $p = 0,378$ e IC $[-329, 866]$, sin rechazar H_0 .

En la muestra complementaria, al exigir partidas disjuntas ($n_1 = n_2 = 1000$), la diferencia muestral aumenta ($\bar{X} - \bar{Y} \approx 996$), y el intervalo de confianza se ensancha ($[-176, 2168]$) al reducirse el tamaño muestral. El p-valor de Welch pasa a 0,096 y el de Mann-Whitney a 0,626 ($r_{rb} \approx 0,03$): el primero se acerca al 5 %, pero no alcanza significación.

En conjunto, la conclusión del contraste principal es robusta: el Cohen's d sigue siendo pequeño ($d \approx 0,02$ frente a $d \approx 0,08$ en la muestra restringida) y, en ambos diseños, el IC incluye el 0. La discrepancia en el p-valor se explica porque se pierde potencia (n menor) y la asignación aleatoria de partidas a cada uno de los roles del contraste incrementa la variabilidad muestral de $\bar{X} - \bar{Y}$.

Por ello, el análisis complementario refuerza la lectura del contraste principal y deja constancia de que la dependencia intra-partida no invalida la decisión de no rechazar H_0 , aunque conviene reconocerla como limitación en el contraste principal.

6.4 Conclusión del Bloque 1

El Bloque 1 queda cubierto con tres contrastes paramétricos diferentes y coherentes con las preguntas de investigación: una t pareada para comparar el oro temprano entre ganador y perdedor, un contraste Z para una proporción poblacional y una t de Welch para comparar medias entre dos roles. En todos los casos se han indicado hipótesis, muestras, supuestos, estadístico de contraste, p-valor, intervalo de confianza e interpretación.

Desde el punto de vista estadístico, el resultado más sólido aparece en el Contraste 1: el equipo ganador presenta una ventaja media de oro al minuto 15 claramente positiva. En cambio, el Contraste 2 no permite afirmar una ventaja significativa del lado azul, y el Contraste 3 no detecta diferencias de daño medio entre ADC y Mid. No todas las creencias de la comunidad son ciertas.

7 Bloque 2: Métodos no paramétricos (30%)

En este bloque se plantean tres contrastes, incluyendo un contraste de bondad de ajuste, un contraste de independencia mediante chi-cuadrado y una comparación de distribuciones mediante Mann-Whitney.

El nivel de significación de referencia será $\alpha = 0,05$. Cuando se añadan contrastes paramétricos complementarios, se interpretarán como análisis de corroboración y no como sustitutos del contraste no paramétrico principal.

7.1 Contraste 1: Bondad de ajuste binomial negativa (kills por equipo)

7.1.1 Planteamiento y contexto

Pregunta: ¿el número de asesinatos por equipo se ajusta razonablemente a una distribución binomial negativa?

En *League of Legends*, los asesinatos (**kills**) son eventos discretos que se acumulan durante la partida. Una primera aproximación para variables de conteo sería la distribución de Poisson, pero existen dos problemas:

1. Poisson asume que la probabilidad de que ocurra una kill es la misma en el minuto 1 que en el minuto 25.
2. Poisson asume que el hecho de que ocurra una kill no afecta a la probabilidad de que ocurra otra.

En una partida real los asesinatos no son totalmente independientes: dependen de la duración, del estado de la partida, de la composición de los equipos, de *teamfights* (peleas entre equipos) y de ventajas de oro acumuladas. Por ello es razonable esperar sobredispersión, es decir, una varianza mayor que la media.

La distribución binomial negativa permite modelar conteos sobredispersos. Este uso es habitual cuando la varianza de una variable de conteo supera claramente a la media, ya que en esos casos una Poisson simple suele quedar demasiado rígida (Payne et al., 2017). Por tanto, con el contexto actual, la distribución binomial negativa es una alternativa más flexible que la Poisson simple.

Antes del contraste filtraremos las partidas con duración inferior a 5 minutos, ya que pueden corresponder a *remakes* o partidas anómalas donde el número de asesinatos no representa una partida desarrollada con normalidad.

Para responder a esta pregunta utilizaremos filas de equipo-partida (**df_team**), restringidas a un único equipo por partida, para evitar que los dos equipos de una misma partida introduzcan dependencia directa entre observaciones. La variable analizada es **kills**, que representa el número total de asesinatos conseguidos por ese equipo.

7.1.2 Hipótesis inicial

Contamos con una muestra formada por una variable aleatoria discreta:

- X_i : número de asesinatos (**kills**) del equipo i , con $i = 1, \dots, n$.

Se dispone de una muestra aleatoria de observaciones $X_1, \dots, X_n$, tras excluir partidas con duración inferior a 5 minutos. Si el modelo binomial negativo fuese adecuado, entonces:

$$X_i \sim \text{NB}(r, p)$$

donde r es el parámetro de tamaño o dispersión y p la probabilidad de éxito en la parametrización utilizada por `scipy.stats.nbinom`.

Es importante destacar que esta parametrización difiere de la interpretación habitual, donde la binomial negativa se define como el número de fracasos antes de obtener un número fijo de

éxitos. En su lugar, se utiliza como una generalización de la distribución de Poisson que permite modelar datos de conteo con sobredispersión.

En esta parametrización:

$$E(X_i) = \frac{r(1-p)}{p}, \quad \text{Var}(X_i) = \frac{r(1-p)}{p^2}$$

Estimaremos r y p por máxima verosimilitud, pero para optimizarla de forma estable, se utiliza la verosimilitud perfilada. Para cada valor de r , el parámetro p queda determinado por la media muestral como (despejando de la fórmula de la media teórica):

$$\hat{p}(r) = \frac{r}{r + \bar{X}}$$

Después se minimiza $-\ell(r, \hat{p}(r))$ respecto de $r > 0$.

En el código, X_i corresponde a la columna `kills` de `df_team_b2c1`.

Se asume independencia entre observaciones de partidas distintas. Esta independencia sigue siendo aproximada, porque un mismo jugador podría aparecer en más de una partida de la muestra, pero se elimina la dependencia directa entre los dos equipos de una misma partida.

7.1.3 Preparación de la muestra

```
df_team_b2c1_duration = df_team[df_team["match_duration_min"] >= 5].copy()
removed_matches_b2c1 = df_team["match_id"].nunique() -
    ↪ df_team_b2c1_duration["match_id"].nunique()
removed_rows_b2c1 = len(df_team) - len(df_team_b2c1_duration)

# Una observación por partida: equipo del lado azul.
df_team_b2c1 = df_team_b2c1_duration[
    df_team_b2c1_duration["side"].str.lower() == "blue"
].copy()
assert df_team_b2c1["match_id"].is_unique, "Debe haber un único equipo azul por
    ↪ partida"

kills_b2c1 = df_team_b2c1["kills"].dropna().astype(int)
n_b2c1 = len(kills_b2c1)
mean_kills_b2c1 = kills_b2c1.mean()
var_kills = kills_b2c1.var(ddof=1)
dispersion_ratio_b2c1 = var_kills / mean_kills_b2c1

def fit_nbinom_mle(x: pd.Series) -> tuple[float, float,
    ↪ optimize.OptimizeResult]:
    """Estima NB(r, p) por EMV usando verosimilitud perfilada en r."""
```

```

x_arr = x.to_numpy(dtype=int)
x_mean = x_arr.mean()

def neg_profile_loglik(log_r: float) -> float:
    r = np.exp(log_r)
    p = r / (r + x_mean)
    return -np.sum(stats.nbinom.logpmf(x_arr, r, p))

result = optimize.minimize_scalar(
    neg_profile_loglik,
    bounds=(np.log(1e-3), np.log(1e4)),
    method="bounded",
    options={"xatol": 1e-10},
)
r_hat = np.exp(result.x)
p_hat = r_hat / (r_hat + x_mean)
return r_hat, p_hat, result

nb_r_hat, nb_p_hat, nb_mle_result = fit_nbinom_mle(kills_b2c1)

print("=== Preparación Bloque 2 - Contraste 1 (kills por equipo) ===")
print(f"Partidas eliminadas por duración < 5 min: {removed_matches_b2c1:,}")
print(f"Filas equipo-partida eliminadas: {removed_rows_b2c1:,}")
print("Muestra final: un equipo por partida (lado azul)")
print(f"Observaciones (n): {n_b2c1:,}")
kills_b2c1.describe().round(2)

```

```

=== Preparación Bloque 2 - Contraste 1 (kills por equipo) ===
Partidas eliminadas por duración < 5 min: 44
Filas equipo-partida eliminadas: 88
Muestra final: un equipo por partida (lado azul)
Observaciones (n): 1,956

```

```

count      1956.00
mean        30.17
std         12.15
min          0.00
25%         21.00
50%         31.00
75%         39.00
max         67.00
Name: kills, dtype: float64

```

```
max_kills = kills_b2c1.max()
values = np.arange(0, max_kills + 1)
obs_counts_raw = kills_b2c1.value_counts().reindex(values,
    ↪ fill_value=0).sort_index()
expected_counts_raw = n_b2c1 * stats.nbinom.pmf(values, nb_r_hat, nb_p_hat)

fig, ax = plt.subplots(figsize=(7.0, 3.5))

ax.bar(
    values,
    obs_counts_raw,
    color="steelblue",
    edgecolor="white",
    alpha=0.85,
    label="Frecuencia observada",
)
ax.plot(
    values,
    expected_counts_raw,
    color="crimson",
    lw=1.8,
    marker="o",
    markersize=3,
    label="Frecuencia esperada binomial negativa",
)

ax.set_title("Kills por equipo: observado vs binomial negativa ajustada",
    ↪ fontsize=11)
ax.set_xlabel("kills", fontsize=10)
ax.set_ylabel("Frecuencia", fontsize=10)
ax.tick_params(labelsize=9)
ax.legend(fontsize=8)

fig.tight_layout(pad=0.6)
plt.show()
```

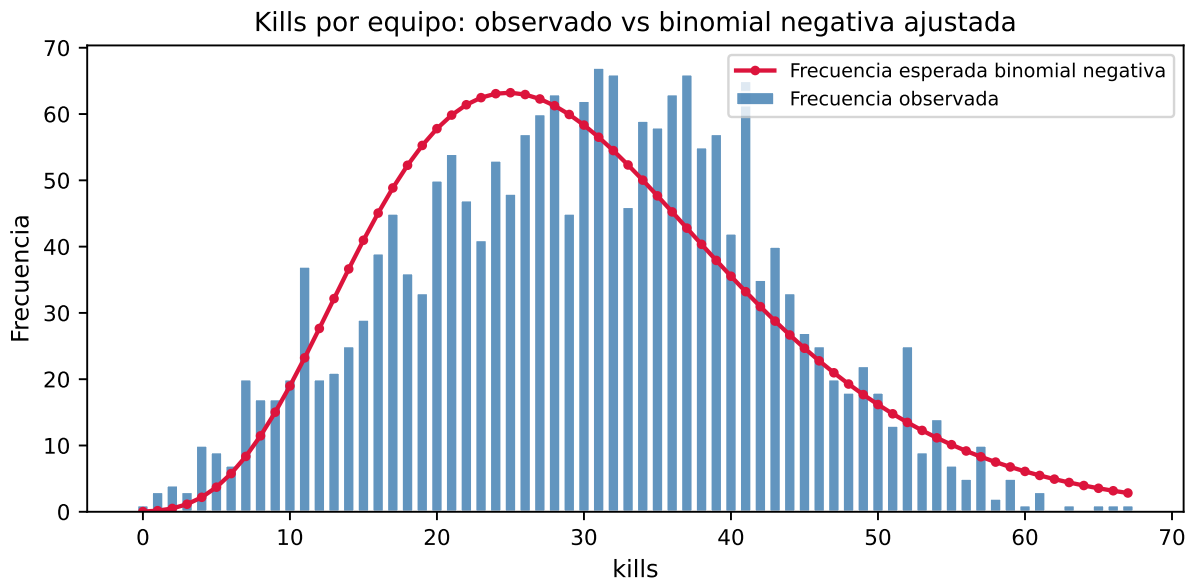


Figure 5: Distribución observada de `kills` por equipo y distribución binomial negativa ajustada.

7.1.4 Verificación de supuestos

```
def build_nbinom_gof_table(
    x: pd.Series,
    r: float,
    p: float,
    min_expected: float = 5,
) -> pd.DataFrame:
    """Agrupa frecuencias observadas y esperadas hasta asegurar esperadas >=
    ↪ min_expected."""

    max_value = int(x.max())
    values = np.arange(0, max_value + 1)
    observed = x.value_counts().reindex(values,
    ↪ fill_value=0).sort_index().to_numpy()
    expected = len(x) * stats.nbinom.pmf(values, r, p)

    # La cola superior no observada se añade a la última categoría.
    expected[-1] += len(x) * stats.nbinom.sf(max_value, r, p)

    groups = []
    current_obs = 0
    current_exp = 0.0
    start = 0

    for value, obs, exp in zip(values, observed, expected):
        if current_exp == 0:
```

```

        start = value
        current_obs += obs
        current_exp += exp
        if current_exp >= min_expected:
            groups.append([start, value, current_obs, current_exp])
            current_obs = 0
            current_exp = 0.0

    if current_exp > 0:
        groups[-1][1] = max_value
        groups[-1][2] += current_obs
        groups[-1][3] += current_exp

    table = pd.DataFrame(groups, columns=["desde", "hasta", "observado",
↪ "esperado"])
    table["esperado"] = table["esperado"] * table["observado"].sum() /
↪ table["esperado"].sum()
    table["residuo_tipificado"] = (table["observado"] - table["esperado"]) /
↪ np.sqrt(table["esperado"])
    table["contribucion_chi2"] = (
        (table["observado"] - table["esperado"]) ** 2 / table["esperado"]
    )
    return table

nbinom_table = build_nbinom_gof_table(kills_b2c1, nb_r_hat, nb_p_hat)
min_expected_b2c1 = nbinom_table["esperado"].min()

print("=== Supuestos Bloque 2 - Contraste 1 (bondad de ajuste binomial
↪ negativa) ===")
print(f"Parámetros NB estimados por EMV: r = {nb_r_hat:.4f}, p =
↪ {nb_p_hat:.4f}")
print(f"Convergencia EMV: {nb_mle_result.success}")
print(f"Grupos tras agrupación (k): {len(nbinom_table)}")
print(f"Frecuencia esperada mínima (>= 5): {min_expected_b2c1:.2f}")
print(f"Sobredispersión s^2 / media = {dispersion_ratio_b2c1:.4f}")
print(f"Primeros 10 grupos de {len(nbinom_table)}:")
nbinom_table.head(10).round(2)

```

```

=== Supuestos Bloque 2 - Contraste 1 (bondad de ajuste binomial negativa) ===
Parámetros NB estimados por EMV: r = 6.2482, p = 0.1716
Convergencia EMV: True
Grupos tras agrupación (k): 60
Frecuencia esperada mínima (>= 5): 5.48
Sobredispersión s^2 / media = 4.8919
Primeros 10 grupos de 60:

```

	desde	hasta	observado	esperado	residuo_tipificado	contribucion_chi2
0	0	5	30	7.74	8.01	64.08
1	6	6	7	5.76	0.52	0.27
2	7	7	20	8.35	4.03	16.25
3	8	8	17	11.46	1.64	2.68
4	9	9	17	15.03	0.51	0.26
5	10	10	20	18.98	0.23	0.05
6	11	11	37	23.23	2.86	8.17
7	12	12	20	27.66	-1.46	2.12
8	13	13	21	32.16	-1.97	3.87
9	14	14	25	36.63	-1.92	3.69

La estimación de los parámetros por EMV es $r \approx 6,25$ y $p \approx 0,172$, con una convergencia exitosa. La razón varianza/media es 4,89, lo que confirma la presencia de sobredispersión y justifica haber pasado de una Poisson simple a una binomial negativa.

Como se estiman dos parámetros (r y p) por EMV a partir de la propia muestra, los grados de libertad son:

$$gl = k - 1 - 2$$

donde k es el número de grupos finales y el último 2 corresponde a los dos parámetros estimados.

7.1.5 Planteamiento del contraste

- H_0 : la variable `kills` por equipo sigue una distribución binomial negativa $NB(r, p)$.
- H_A : la variable `kills` por equipo no sigue una distribución binomial negativa $NB(r, p)$.

7.1.6 Estadístico de contraste

El estadístico de bondad de ajuste es:

$$\chi_{\text{obs}}^2 = \sum_{j=1}^k \frac{(O_j - E_j)^2}{E_j}$$

donde O_j es la frecuencia observada en el grupo j y E_j la frecuencia esperada bajo la distribución binomial negativa ajustada.

Bajo H_0 y con las frecuencias esperadas agrupadas correctamente:

$$\chi_{\text{obs}}^2 \sim \chi_{k-3}^2$$

7.1.7 Contraste inferencial

```

observed_b2c1 = nbinom_table["observado"].to_numpy(dtype=float)
expected_b2c1 = nbinom_table["esperado"].to_numpy(dtype=float)

chi2_b2c1 = ((observed_b2c1 - expected_b2c1) ** 2 / expected_b2c1).sum()
df_b2c1 = len(nbinom_table) - 3
p_b2c1 = stats.chi2.sf(chi2_b2c1, df_b2c1)

print("=== Bloque 2 - Contraste 1: bondad de ajuste binomial negativa ===")
print(f"chi2({df_b2c1}) = {chi2_b2c1:.4f} | p-valor = {p_b2c1:.2e}")

top_contrib_b2c1 = nbinom_table.sort_values("contribucion_chi2",
↪ ascending=False).head(8)
print("\nGrupos con mayor contribución al estadístico chi-cuadrado:")
print(top_contrib_b2c1.round(2).to_string(index=False))

nbinom_table.round(2)

```

```

=== Bloque 2 - Contraste 1: bondad de ajuste binomial negativa ===
chi2(57) = 287.8460 | p-valor = 1.50e-32

```

Grupos con mayor contribución al estadístico chi-cuadrado:

	desde	hasta	observado	esperado	residuo_tipificado	contribucion_chi2
	0	5	30	7.74	8.01	64.08
	41	41	65	33.21	5.52	30.44
	66	67	2	28.15	-4.93	24.29
	7	7	20	8.35	4.03	16.25
	37	37	66	42.79	3.55	12.59
	52	52	25	13.48	3.14	9.84
	39	39	57	37.92	3.10	9.60
	19	19	33	55.26	-2.99	8.97

	desde	hasta	observado	esperado	residuo_tipificado	contribucion_chi2
0	0	5	30	7.74	8.01	64.08
1	6	6	7	5.76	0.52	0.27
2	7	7	20	8.35	4.03	16.25
3	8	8	17	11.46	1.64	2.68
4	9	9	17	15.03	0.51	0.26
5	10	10	20	18.98	0.23	0.05
6	11	11	37	23.23	2.86	8.17
7	12	12	20	27.66	-1.46	2.12
8	13	13	21	32.16	-1.97	3.87
9	14	14	25	36.63	-1.92	3.69
10	15	15	29	40.96	-1.87	3.49

	desde	hasta	observado	esperado	residuo_tipificado	contribucion_chi2
11	16	16	39	45.06	-0.90	0.82
12	17	17	45	48.85	-0.55	0.30
13	18	18	36	52.27	-2.25	5.07
14	19	19	33	55.26	-2.99	8.97
15	20	20	50	57.79	-1.03	1.05
16	21	21	54	59.84	-0.76	0.57
17	22	22	47	61.40	-1.84	3.38
18	23	23	41	62.47	-2.72	7.38
19	24	24	53	63.07	-1.27	1.61
20	25	25	48	63.22	-1.91	3.66
21	26	26	57	62.94	-0.75	0.56
22	27	27	60	62.28	-0.29	0.08
23	28	28	63	61.26	0.22	0.05
24	29	29	45	59.93	-1.93	3.72
25	30	30	62	58.33	0.48	0.23
26	31	31	67	56.51	1.40	1.95
27	32	32	66	54.49	1.56	2.43
28	33	33	46	52.32	-0.87	0.76
29	34	34	59	50.03	1.27	1.61
30	35	35	58	47.66	1.50	2.24
31	36	36	63	45.24	2.64	6.97
32	37	37	66	42.79	3.55	12.59
33	38	38	55	40.35	2.31	5.32
34	39	39	57	37.92	3.10	9.60
35	40	40	42	35.54	1.08	1.18
36	41	41	65	33.21	5.52	30.44
37	42	42	35	30.95	0.73	0.53
38	43	43	40	28.76	2.09	4.39
39	44	44	33	26.67	1.23	1.50
40	45	45	27	24.67	0.47	0.22
41	46	46	25	22.77	0.47	0.22
42	47	47	20	20.97	-0.21	0.04
43	48	48	18	19.27	-0.29	0.08
44	49	49	22	17.67	1.03	1.06
45	50	50	18	16.18	0.45	0.21
46	51	51	13	14.78	-0.46	0.21
47	52	52	25	13.48	3.14	9.84
48	53	53	9	12.27	-0.93	0.87
49	54	54	14	11.16	0.85	0.73
50	55	55	7	10.12	-0.98	0.96
51	56	56	5	9.17	-1.38	1.90
52	57	57	10	8.30	0.59	0.35
53	58	58	2	7.50	-2.01	4.03
54	59	59	5	6.76	-0.68	0.46
55	60	60	1	6.09	-2.06	4.26
56	61	61	3	5.48	-1.06	1.12
57	62	63	1	9.34	-2.73	7.45

	desde	hasta	observado	esperado	residuo_tipificado	contribucion_chi2
58	64	65	1	7.51	-2.38	5.64
59	66	67	2	28.15	-4.93	24.29

El p-valor ≈ 0 nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que `kills` por equipo siga una distribución binomial negativa, es $\approx 0\%$.

Si H_0 fuese cierta, la probabilidad de observar un estadístico de contraste tan extremo como $\chi_{\text{obs}}^2 = 287,85$ (con $gl = 57$) sería del orden de 10^{-32} . Por tanto, rechazamos H_0 : en esta muestra, `kills` no se ajusta de forma adecuada a una binomial negativa $NB(r, p)$ con los parámetros estimados por EMV.

La tabla de contribuciones indica que parte importante del rechazo procede de los valores muy bajos de `kills` (por ejemplo, el grupo 0-5 asesinatos), donde se observan muchas más frecuencias de las esperadas por el modelo. En términos prácticos, esto sugiere que no basta con introducir sobredispersión: la distribución de asesinatos está afectada por la duración, el ritmo de partida y partidas desequilibradas.

Por ello, `kills` debe tratarse como variable de conteo sobredispersa, pero una binomial negativa con estos valores de r y p no es suficiente para explicar su distribución.

7.2 Contraste 2: Independencia entre clase de campeón y resultado

7.2.1 Planteamiento y contexto

Pregunta: ¿la clase de campeón es independiente del resultado de la partida?

En *League of Legends*, los campeones pueden agruparse en clases: asesinos, luchadores, magos, tiradores, soportes o tanques. Estas clases condicionan el estilo de juego, el tipo de daño, la supervivencia y la utilidad que un jugador aporta al equipo.

En la comunidad se suele discutir si ciertas clases de campeones son más efectivas que otras para ganar partidas.

Para responder a esta pregunta utilizaremos filas de jugador-partida (`df`). Cada observación corresponde a un jugador con una clase de campeón y un resultado (`win = 1` victoria, `win = 0` derrota).

7.2.2 Hipótesis inicial

Contamos con dos variables categóricas:

- X : clase de campeón (`champion_class`).
- Y : resultado de la partida para el jugador (`win`).

Se dispone de una tabla de contingencia con las frecuencias observadas O_{ij} para cada combinación entre clase de campeón y resultado. Bajo independencia, las frecuencias esperadas se calculan como:

$$E_{ij} = \frac{n_{i.}n_{.j}}{n}$$

En el código, X corresponde a `champion_class` y Y a `win`. Eliminamos la clase `Unknown` para que la interpretación se limite a clases funcionales definidas.

Se asume independencia entre observaciones, aunque debe considerarse aproximada porque diez jugadores proceden de una misma partida y un mismo jugador puede aparecer en varias partidas. Más adelante se podría repetir el contraste con un jugador aleatorio por equipo-partida para evaluar la sensibilidad ante esta dependencia.

7.2.3 Preparación de la muestra

```
df_b2c2 = df[df["champion_class"].notna() & (df["champion_class"] !=
↪ "Unknown")].copy()
table_b2c2 = pd.crosstab(df_b2c2["champion_class"], df_b2c2["win"])
table_b2c2 = table_b2c2.rename(columns={0: "Derrota", 1: "Victoria"})

def add_total_margins(table: pd.DataFrame) -> pd.DataFrame:
    """Añade fila y columna TOTAL solo para presentación."""

    table_with_total = table.copy()
    table_with_total["TOTAL"] = table_with_total.sum(axis=1)
    total_row = table_with_total.sum(axis=0).to_frame().T
    total_row.index = ["TOTAL"]
    return pd.concat([table_with_total, total_row])

print(f"Observaciones jugador-partida: {len(df_b2c2):,}")
print(f"Clases de campeón consideradas: {table_b2c2.shape[0]}")
add_total_margins(table_b2c2)
```

Observaciones jugador-partida: 19,926

Clases de campeón consideradas: 6

win	Derrota	Victoria	TOTAL
Assassin	877	833	1710
Fighter	2832	2926	5758
Mage	1902	1978	3880
Marksman	2162	2166	4328

win	Derrota	Victoria	TOTAL
Support	912	887	1799
Tank	1271	1180	2451
TOTAL	9956	9970	19926

```
prop_b2c2 = table_b2c2.div(table_b2c2.sum(axis=1), axis=0)

fig, ax = plt.subplots(figsize=(7.0, 3.8))
prop_b2c2[["Derrota", "Victoria"]].plot(
    kind="bar",
    stacked=True,
    color=["lightcoral", "steelblue"],
    edgecolor="white",
    alpha=0.9,
    ax=ax,
)

ax.axhline(0.5, color="gray", ls=":", lw=1.2)
ax.set_title("Resultado por clase de campeón", fontsize=11)
ax.set_xlabel("Clase de campeón", fontsize=10)
ax.set_ylabel("Proporción", fontsize=10)
ax.tick_params(axis="x", labelrotation=35, labelsz=9)
ax.tick_params(axis="y", labelsz=9)
ax.legend(fontsize=8, loc="upper right")

fig.tight_layout(pad=0.6)
plt.show()
```

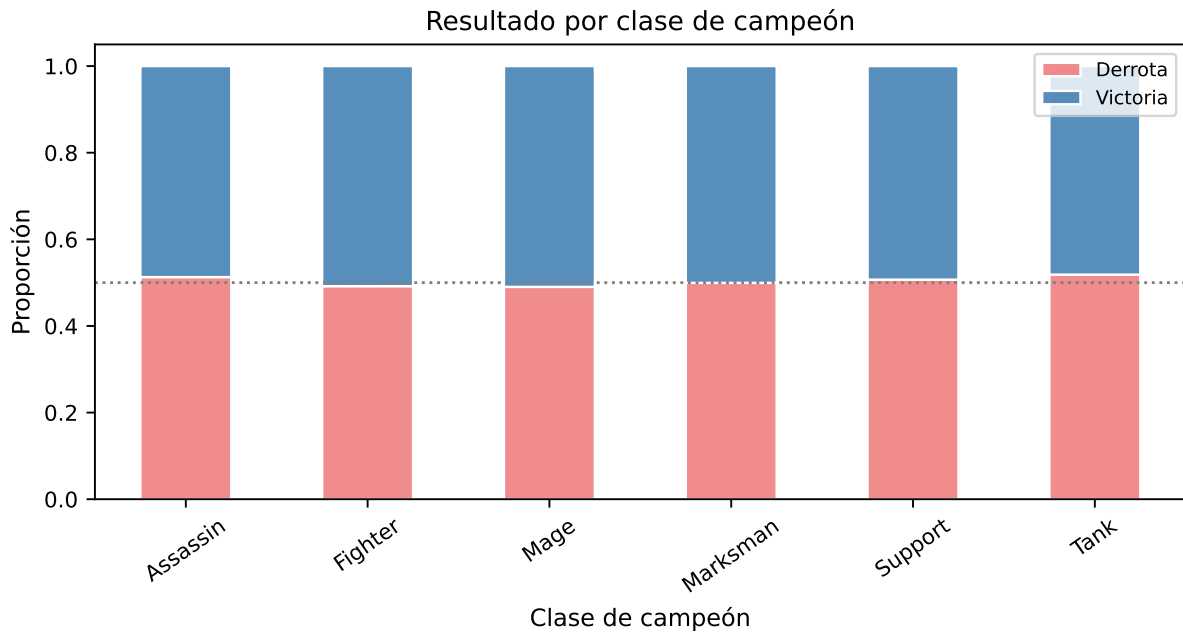


Figure 6: Proporción de victorias y derrotas por clase de campeón.

7.2.4 Verificación de supuestos

```
chi2_b2c2, p_b2c2, dof_b2c2, expected_b2c2 = stats.chi2_contingency(table_b2c2)
expected_table_b2c2 = pd.DataFrame(
    expected_b2c2,
    index=table_b2c2.index,
    columns=table_b2c2.columns,
)

print("=== Supuestos Bloque 2 - Contraste 2 (chi-cuadrado de independencia)
↳ ===")
print(f"Frecuencia esperada mínima: {expected_table_b2c2.min().min():.2f}")
add_total_margins(expected_table_b2c2).round(2)
```

```
=== Supuestos Bloque 2 - Contraste 2 (chi-cuadrado de independencia) ===
Frecuencia esperada mínima: 854.40
```

win	Derrota	Victoria	TOTAL
Assassin	854.40	855.60	1710.0
Fighter	2876.98	2881.02	5758.0
Mage	1938.64	1941.36	3880.0
Marksman	2162.48	2165.52	4328.0
Support	898.87	900.13	1799.0

win	Derrota	Victoria	TOTAL
Tank	1224.64	1226.36	2451.0
TOTAL	9956.00	9970.00	19926.0

Todas las frecuencias esperadas son mayores que 5, por lo que podemos aplicar el contraste chi-cuadrado de independencia.

7.2.5 Planteamiento del contraste

- H_0 : `champion_class` y `win` son independientes.
- H_A : `champion_class` y `win` no son independientes.

7.2.6 Estadístico de contraste

El estadístico del contraste es:

$$\chi_{\text{obs}}^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

Bajo H_0 :

$$\chi_{\text{obs}}^2 \sim \chi_{(r-1)(c-1)}^2$$

donde r es el número de clases de campeón y c el número de categorías del resultado.

7.2.7 Contraste inferencial

```
n_b2c2 = table_b2c2.to_numpy().sum()
rows_b2c2, cols_b2c2 = table_b2c2.shape
cramer_v_b2c2 = np.sqrt(chi2_b2c2 / (n_b2c2 * min(rows_b2c2 - 1, cols_b2c2 -
↪ 1)))

print("=== Bloque 2 - Contraste 2: independencia champion_class vs win ===")
print(f"chi2({dof_b2c2}) = {chi2_b2c2:.4f} | p-valor = {p_b2c2:.4f}")
print(f"V de Cramér = {cramer_v_b2c2:.4f}")
```

```
=== Bloque 2 - Contraste 2: independencia champion_class vs win ===
chi2(5) = 7.8753 | p-valor = 0.1632
V de Cramér = 0.0199
```

El p -valor = 0,1632 nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que la clase de campeón y el resultado sean independientes, es del 16,32%. Si rechazásemos H_0 , cometeríamos un error de tipo I en el 16,32% de los casos.

Si H_0 fuese cierta, la probabilidad de obtener un estadístico de contraste tan extremo como $\chi^2_{\text{obs}} = 7,88$ sería del 16,32%. Por tanto, no rechazamos H_0 y atribuimos las discrepancias observadas al azar.

La V de Cramer es 0,0199, lo que indica una asociación prácticamente nula entre clase de campeón y resultado.

En términos prácticos, en esta muestra no podemos afirmar que una clase de campeón concreta tenga una tasa de victoria significativamente distinta. Esto no implica que la composición sea irrelevante, sino que la clase del campeón, por sí sola, no explica de forma apreciable la victoria individual en estas partidas.

Por último, esta conclusión debe interpretarse con cautela: las clases son categorías amplias y no recogen diferencias entre campeones concretos, parches, composición completa ni nivel de ejecución del jugador.

7.2.8 Contraste complementario: sensibilidad ante dependencia intra-partida

En el contraste principal, cada partida aporta hasta diez observaciones (cinco jugadores por equipo). Todas comparten el mismo `match_id`, de modo que el resultado `win` es idéntico dentro de cada equipo y el contexto de la partida se repite en varias filas. Por ello, la independencia asumida en el ² de independencia es solo aproximada.

Para comprobar si esta limitación altera la conclusión, repetimos el mismo contraste (H_0 y H_A idénticos) sobre una muestra restringida: un jugador aleatorio por (`match_id`, `team_id`), con `random_state = 42`. Así cada equipo contribuye con una sola fila (clase del campeón elegido y victoria o derrota del equipo).

```
def build_one_player_per_team(
    player_df: pd.DataFrame,
    seed: int = 42,
) -> pd.DataFrame:
    """Un jugador aleatorio por (match_id, team_id) con clase y resultado del
    ↪ equipo."""

    subset = player_df.loc[
        player_df["champion_class"].notna() & (player_df["champion_class"] !=
    ↪ "Unknown"),
        ["match_id", "team_id", "champion_class", "win"],
    ].copy()

    rng = np.random.default_rng(seed)
    subset["_rand"] = rng.random(len(subset))

    one_per_team = (
        subset.sort_values(["match_id", "team_id", "_rand"])
```

```

        .drop_duplicates(["match_id", "team_id"], keep="first")
        .drop(columns="_rand")
    )

    return one_per_team

df_b2c2_team = build_one_player_per_team(df)
table_b2c2_team = pd.crosstab(df_b2c2_team["champion_class"],
    ↪ df_b2c2_team["win"])
table_b2c2_team = table_b2c2_team.rename(columns={0: "Derrota", 1: "Victoria"})

n_b2c2_team = len(df_b2c2_team)
n_matches_b2c2_team = df_b2c2_team["match_id"].nunique()

b2c2_sample_report = pd.DataFrame(
    {
        "Principal (todas las filas)": [len(df_b2c2),
            ↪ df_b2c2["match_id"].nunique()],
        "Un jugador por equipo-partida": [n_b2c2_team, n_matches_b2c2_team],
    },
    index=["n observaciones", "n partidas"],
)

print("=== Muestra complementaria (un jugador por equipo-partida) ===")
print(b2c2_sample_report.to_string())
print(f"Equipos-partida esperados (2 por partida): {n_matches_b2c2_team *
    ↪ 2:,}")
assert n_b2c2_team == df_b2c2_team.groupby(["match_id", "team_id"]).ngroups
add_total_margins(table_b2c2_team)

```

```

=== Muestra complementaria (un jugador por equipo-partida) ===
              Principal (todas las filas)  Un jugador por equipo-partida
n observaciones                19926                4000
n partidas                    2000                2000
Equipos-partida esperados (2 por partida): 4,000

```

win	Derrota	Victoria	TOTAL
Assassin	190	173	363
Fighter	539	563	1102
Mage	363	405	768
Marksman	435	447	882
Support	219	177	396
Tank	254	235	489
TOTAL	2000	2000	4000

```

prop_b2c2_team = table_b2c2_team.div(table_b2c2_team.sum(axis=1), axis=0)

fig, ax = plt.subplots(figsize=(7.0, 3.8))
prop_b2c2_team[["Derrota", "Victoria"]].plot(
    kind="bar",
    stacked=True,
    color=["lightcoral", "steelblue"],
    edgecolor="white",
    alpha=0.9,
    ax=ax,
)

ax.axhline(0.5, color="gray", ls=":", lw=1.2)
ax.set_title("Resultado por clase (muestra complementaria)", fontsize=11)
ax.set_xlabel("Clase de campeón", fontsize=10)
ax.set_ylabel("Proporción", fontsize=10)
ax.tick_params(axis="x", labelrotation=35, labelsz=9)
ax.tick_params(axis="y", labelsz=9)
ax.legend(fontsize=8, loc="upper right")

fig.tight_layout(pad=0.6)
plt.show()

```

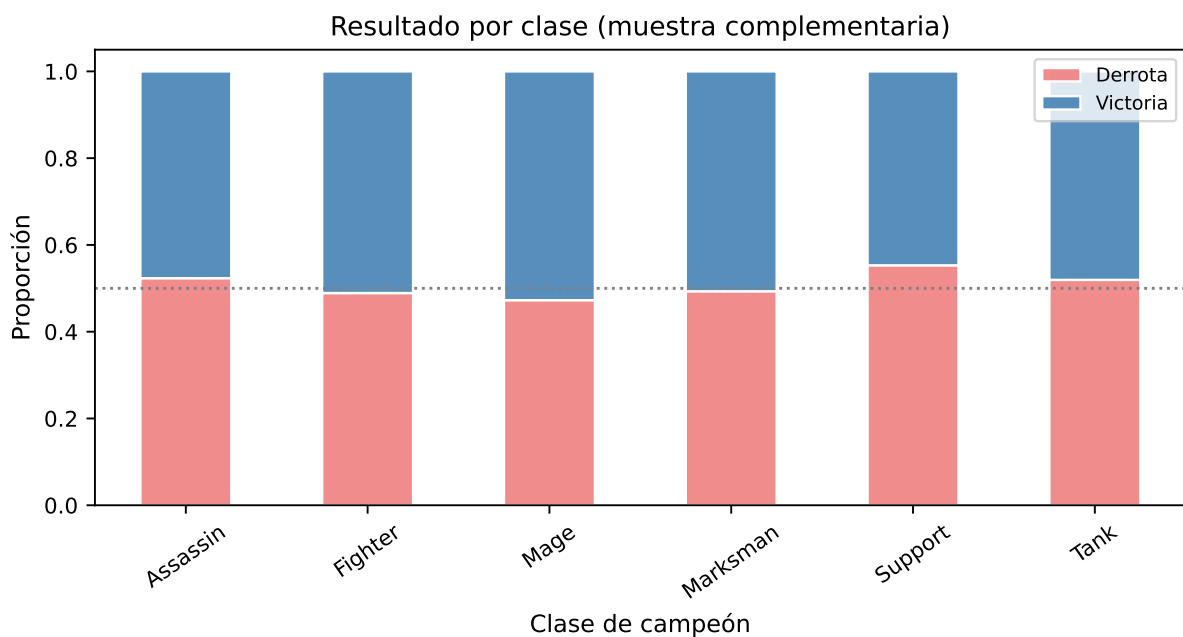


Figure 7: Proporción de victorias y derrotas por clase de campeón en la muestra complementaria (un jugador por equipo-partida).

Se aplican los mismos procedimientos del contraste principal (χ^2 de independencia + V de Cramér).

```

def chi2_independence_summary(table: pd.DataFrame) -> dict:
    chi2, p_val, dof, expected = stats.chi2_contingency(table)
    expected_df = pd.DataFrame(expected, index=table.index,
    ↪ columns=table.columns)
    n = table.to_numpy().sum()
    rows, cols = table.shape
    cramer = np.sqrt(chi2 / (n * min(rows - 1, cols - 1)))
    return {
        "n": int(n),
        "chi2": chi2,
        "dof": dof,
        "p_valor": p_val,
        "Cramér_V": cramer,
        "min_esperada": expected_df.min().min(),
    }

chi2_b2c2_team, p_b2c2_team, dof_b2c2_team, expected_b2c2_team =
    ↪ stats.chi2_contingency(table_b2c2_team)
expected_table_b2c2_team = pd.DataFrame(
    expected_b2c2_team,
    index=table_b2c2_team.index,
    columns=table_b2c2_team.columns,
)

summary_b2c2_main = chi2_independence_summary(table_b2c2)
summary_b2c2_team = chi2_independence_summary(table_b2c2_team)

compare_b2c2 = pd.DataFrame(
    {"Contraste principal": summary_b2c2_main, "Un jugador por equipo-partida":
    ↪ summary_b2c2_team}
)
print("=== Comparación Contraste 2 principal vs sensibilidad ===")
print(compare_b2c2.round(4).to_string())

```

```

=== Comparación Contraste 2 principal vs sensibilidad ===
               Contraste principal  Un jugador por equipo-partida
n                19926.0000                4000.0000
chi2                 7.8753                 8.9718
dof                  5.0000                 5.0000
p_valor              0.1632                 0.1102
Cramér_V             0.0199                 0.0474
min_esperada         854.3993                181.5000

```

En el contraste principal (todas las filas jugador-partida, con posible dependencia intra-partida) no se rechazó H_0 y la V de Cramér fue prácticamente nula, como se recoge en la tabla comparativa.

En la muestra complementaria, con un jugador aleatorio por equipo-partida ($n = 4000$), el estadístico aumenta ligeramente ($\chi^2_{\text{obs}} \approx 8,97$, $p \approx 0,110$) y la V de Cramér pasa a 0,047. El p-valor sigue siendo superior al 5 %, por lo que tampoco se rechaza H_0 en este contraste complementario.

En conjunto, la conclusión del contraste principal es robusta en dirección: en ambos diseños no se rechaza H_0 y la asociación entre clase de campeón y resultado permanece débil (V de Cramér $< 0,05$). La diferencia moderada entre estadísticos se explica porque el submuestreo reduce el tamaño muestral efectivo.

Por ello, el análisis complementario refuerza la lectura del contraste principal y deja constancia de que la dependencia intra-partida no invalida la decisión de no rechazar H_0 , aunque conviene reconocerla como limitación del contraste principal.

7.3 Contraste 3: Mann-Whitney (daño por minuto entre regiones)

7.3.1 Planteamiento y contexto

Pregunta: ¿El estilo de juego en Corea (KR) es más agresivo que en Europa (EUW)?

La comparación entre regiones es habitual en la comunidad competitiva. Corea (KR) suele asociarse con un estilo más agresivo, mientras que Europa (EUW) presenta un juego más variado en ritmo.

Para responder a esta pregunta utilizaremos filas de equipo-partida (`df_team`), separando las observaciones de EUW y KR. La variable analizada es `damage_per_min`, que resume el daño infligido por minuto y permite comparar partidas de distinta duración. Cuanto mayor sea el daño por minuto, más agresivo es el estilo de juego.

Antes del contraste filtraremos las partidas con duración inferior a 5 minutos, ya que pueden corresponder a *remakes* o partidas anómalas donde el daño por minuto no representa una partida desarrollada con normalidad. Este filtro es especialmente importante aquí porque `damage_per_min` divide el daño total entre la duración: en partidas extremadamente cortas, valores atípicos pueden distorsionar la comparación entre regiones.

7.3.2 Hipótesis inicial

Contamos con dos muestras independientes:

- X_i : daño por minuto de equipos de EUW.
- Y_j : daño por minuto de equipos de KR.

Se dispone de dos muestras aleatorias $X_1, \dots, X_{n_1}$ e $Y_1, \dots, Y_{n_2}$, tras excluir partidas con duración inferior a 5 minutos.

En el código, X_i corresponde a `damage_per_min` para `region == "EUW"` y Y_j a `damage_per_min` para `region == "KR"`.

Se asume independencia entre observaciones de equipo-partida. No obstante, esta independencia es aproximada, ya que los dos equipos de una misma partida comparten contexto.

7.3.3 Preparación de la muestra

```
df_b2c3_duration = df_team[df_team["match_duration_min"] >= 5].copy()
removed_matches_b2c3 = df_team["match_id"].nunique() -
    ↪ df_b2c3_duration["match_id"].nunique()
removed_rows_b2c3 = len(df_team) - len(df_b2c3_duration)

df_b2c3 = (
    df_b2c3_duration[df_b2c3_duration["region"].isin(["EUW", "KR"])]
    .dropna(subset=["damage_per_min"])
    .copy()
)

euw_b2c3 = df_b2c3.loc[df_b2c3["region"] == "EUW",
    ↪ "damage_per_min"].astype(float)
kr_b2c3 = df_b2c3.loc[df_b2c3["region"] == "KR",
    ↪ "damage_per_min"].astype(float)

n_euw_b2c3 = len(euw_b2c3)
n_kr_b2c3 = len(kr_b2c3)

print(f"Partidas eliminadas por duración < 5 min: {removed_matches_b2c3:,}")
print(f"Filas equipo-partida eliminadas: {removed_rows_b2c3:,}")
print(f"Observaciones EUW: {n_euw_b2c3:,}")
print(f"Observaciones KR: {n_kr_b2c3:,}")
pd.DataFrame({"EUW": euw_b2c3.describe(), "KR": kr_b2c3.describe()}).round(2)
```

Partidas eliminadas por duración < 5 min: 44

Filas equipo-partida eliminadas: 88

Observaciones EUW: 1,954

Observaciones KR: 1,958

	EUW	KR
count	1954.00	1958.00
mean	3620.81	3448.92
std	904.11	886.22
min	785.69	974.90
25%	3009.76	2837.08
50%	3643.96	3486.53
75%	4236.80	4053.47
max	7443.90	6546.93

```
x_min = min(euw_b2c3.min(), kr_b2c3.min())
x_max = max(euw_b2c3.max(), kr_b2c3.max())
```

```

bins = np.linspace(x_min, x_max, 41)

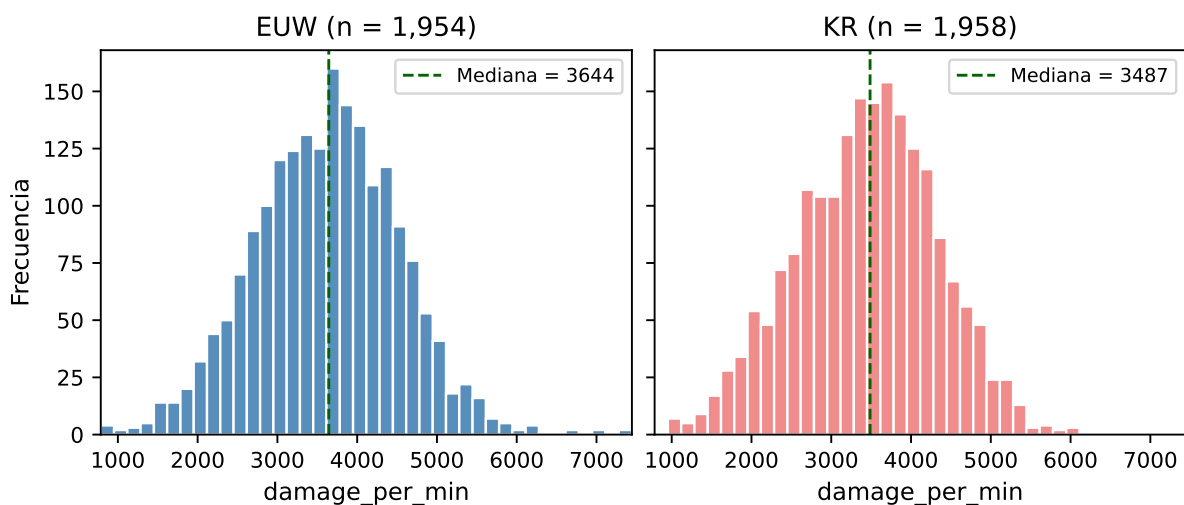
fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.5), sharex=True, sharey=True)

for ax, region, color, data in zip(
    axes,
    ["EUW", "KR"],
    ["steelblue", "lightcoral"],
    [euw_b2c3, kr_b2c3],
):
    ax.hist(data, bins=bins, color=color, edgecolor="white", alpha=0.9)
    ax.axvline(
        data.median(),
        color="darkgreen",
        ls="--",
        lw=1.2,
        label=f"Mediana = {data.median():.0f}",
    )
    ax.set_title(f"{region} (n = {len(data):,})", fontsize=11)
    ax.set_xlabel("damage_per_min", fontsize=10)
    ax.tick_params(labelsize=9)
    ax.legend(fontsize=8, loc="upper right")

axes[0].set_ylabel("Frecuencia", fontsize=10)
axes[0].set_xlim(x_min, x_max)
fig.suptitle("Daño por minuto por región", fontsize=11, y=0.98)
fig.tight_layout(rect=[0, 0, 1, 0.94], pad=0.6)
plt.show()

```

Daño por minuto por región

Figure 8: Distribución muestral de `damage_per_min` por región.

7.3.4 Verificación de supuestos

```
dag_euw_b2c3 = stats.normaltest(euw_b2c3)
dag_kr_b2c3 = stats.normaltest(kr_b2c3)
levene_b2c3 = stats.levene(euw_b2c3, kr_b2c3, center="median")

print("=== Supuestos Bloque 2 - Contraste 3 (Mann-Whitney) ===")
print(f"n_EUW = {n_euw_b2c3:,} | n_KR = {n_kr_b2c3:,}")
print(f"D'Agostino-Pearson EUW: K2 = {dag_euw_b2c3.statistic:.4f}, p =
↪ {dag_euw_b2c3.pvalue:.2e}")
print(f"D'Agostino-Pearson KR: K2 = {dag_kr_b2c3.statistic:.4f}, p =
↪ {dag_kr_b2c3.pvalue:.2e}")
print(f"Levene (dispersión): stat = {levene_b2c3.statistic:.4f}, p =
↪ {levene_b2c3.pvalue:.2e}")
```

```
=== Supuestos Bloque 2 - Contraste 3 (Mann-Whitney) ===
n_EUW = 1,954 | n_KR = 1,958
D'Agostino-Pearson EUW: K2 = 8.7382, p = 1.27e-02
D'Agostino-Pearson KR: K2 = 4.4713, p = 1.07e-01
Levene (dispersión): stat = 0.1643, p = 6.85e-01
```

Se rechaza la normalidad en EUW con D'Agostino-Pearson, mientras que en KR no se rechaza al 5 %. El contraste principal será la prueba U de Mann-Whitney, que compara rangos y no exige normalidad.

Sin embargo, también usaremos contrastes paramétricos complementarios de diferencia de medias para corroborar la conclusión. Los tamaños muestrales son elevados en ambas regiones ($n_1 \approx n_2 \approx 2000$), por lo que el Teorema Central del Límite justifica la aproximación normal de las medias muestrales incluso si las variables originales no son estrictamente normales.

El test de Levene no detecta diferencias significativas de dispersión, por lo que la t de Student con varianzas iguales es admisible. Además, aplicaremos la t de Welch, que no asume igualdad de varianzas, como versión robusta. Si ambas pruebas coinciden con Mann-Whitney, la conclusión será más estable.

7.3.5 Planteamiento del contraste

- H_0 : las distribuciones de `damage_per_min` en EUW y KR son iguales.
- H_A : las distribuciones de `damage_per_min` en EUW y KR son distintas.

7.3.6 Estadístico de contraste

La prueba U de Mann-Whitney ordena conjuntamente las observaciones de ambos grupos y compara la suma de rangos. Si las distribuciones son similares, los rangos deberían mezclarse de forma equilibrada entre EUW y KR.

Como medida de tamaño del efecto utilizaremos la correlación biserial de rangos:

$$r_{rb} = \frac{2U}{n_1 n_2} - 1$$

7.3.7 Contrastes complementarios paramétricos: t de Student y t de Welch

Los contrastes paramétricos complementarios comparan las medias poblacionales de `damage_per_min` entre regiones:

- $H_0 : \mu_{\text{EUW}} - \mu_{\text{KR}} = 0$
- $H_A : \mu_{\text{EUW}} - \mu_{\text{KR}} \neq 0$

El contraste t de Student para muestras independientes asume igualdad de varianzas poblacionales. Su estadístico es:

$$t = \frac{\bar{X} - \bar{Y}}{S_p \sqrt{\frac{1}{n_X} + \frac{1}{n_Y}}}$$

donde S_p es la desviación típica combinada:

$$S_p = \sqrt{\frac{(n_X - 1)S_X^2 + (n_Y - 1)S_Y^2}{n_X + n_Y - 2}}$$

Como contraste adicional, usamos también la t de Welch, que no asume varianzas iguales:

$$t = \frac{\bar{X} - \bar{Y}}{\sqrt{\frac{S_X^2}{n_X} + \frac{S_Y^2}{n_Y}}}$$

donde $\bar{X}$ y $\bar{Y}$ son las medias muestrales de EUW y KR, respectivamente. ##### Contraste inferencial

```
u_b2c3, p_b2c3_mw = stats.mannwhitneyu(euw_b2c3, kr_b2c3,
    ↪ alternative="two-sided")
rank_biserial_b2c3 = 2 * u_b2c3 / (n_euw_b2c3 * n_kr_b2c3) - 1

# Contrastes paramétricos complementarios: Student (varianzas iguales) y Welch
t_student_b2c3, p_b2c3_student = stats.ttest_ind(euw_b2c3, kr_b2c3,
    ↪ equal_var=True)
t_welch_b2c3, p_b2c3_welch = stats.ttest_ind(euw_b2c3, kr_b2c3,
    ↪ equal_var=False)
xbar_b2c3 = euw_b2c3.mean()
ybar_b2c3 = kr_b2c3.mean()
diff_mean_b2c3 = xbar_b2c3 - ybar_b2c3
```

```

s_euw_b2c3 = euw_b2c3.std(ddof=1)
s_kr_b2c3 = kr_b2c3.std(ddof=1)

df_student_b2c3 = n_euw_b2c3 + n_kr_b2c3 - 2
sp_b2c3 = np.sqrt(
    ((n_euw_b2c3 - 1) * s_euw_b2c3**2 + (n_kr_b2c3 - 1) * s_kr_b2c3**2)
    / df_student_b2c3
)
se_student_b2c3 = sp_b2c3 * np.sqrt(1 / n_euw_b2c3 + 1 / n_kr_b2c3)
tcrit_student_b2c3 = stats.t.ppf(0.975, df_student_b2c3)
ci_low_student_b2c3 = diff_mean_b2c3 - tcrit_student_b2c3 * se_student_b2c3
ci_high_student_b2c3 = diff_mean_b2c3 + tcrit_student_b2c3 * se_student_b2c3

se_welch_b2c3 = np.sqrt(s_euw_b2c3**2 / n_euw_b2c3 + s_kr_b2c3**2 / n_kr_b2c3)
df_welch_b2c3 = (
    (s_euw_b2c3**2 / n_euw_b2c3 + s_kr_b2c3**2 / n_kr_b2c3) ** 2
    / (
        (s_euw_b2c3**2 / n_euw_b2c3) ** 2 / (n_euw_b2c3 - 1)
        + (s_kr_b2c3**2 / n_kr_b2c3) ** 2 / (n_kr_b2c3 - 1)
    )
)
tcrit_b2c3 = stats.t.ppf(0.975, df_welch_b2c3)
ci_low_b2c3 = diff_mean_b2c3 - tcrit_b2c3 * se_welch_b2c3
ci_high_b2c3 = diff_mean_b2c3 + tcrit_b2c3 * se_welch_b2c3
cohen_d_b2c3 = diff_mean_b2c3 / sp_b2c3

print("=== Bloque 2 - Contraste 3: Mann-Whitney EUW vs KR ===")
print(f"Mediana EUW = {euw_b2c3.median():.2f} | Mediana KR =
    ↪ {kr_b2c3.median():.2f}")
print(f"Media EUW = {euw_b2c3.mean():.2f} | Media KR = {kr_b2c3.mean():.2f}")
print(f"U = {u_b2c3:.0f} | p-valor(bilateral) = {p_b2c3_mw:.2e}")
print(f"r biserial de rangos = {rank_biserial_b2c3:.4f}")

print("\n=== Contrastes complementarios paramétricos ===")
print(f"Diferencia de medias EUW - KR = {diff_mean_b2c3:.2f}")
print(
    f"Student var. iguales: t({df_student_b2c3}) = {t_student_b2c3:.4f} | "
    f"p-valor = {p_b2c3_student:.2e} | "
    f"IC 95% = [{ci_low_student_b2c3:.2f}, {ci_high_student_b2c3:.2f}]"
)
print(
    f"Welch var. desiguales: t({df_welch_b2c3:.1f}) = {t_welch_b2c3:.4f} | "
    f"p-valor = {p_b2c3_welch:.2e} | "
    f"IC 95% = [{ci_low_b2c3:.2f}, {ci_high_b2c3:.2f}]"
)
print(f"Cohen's d = {cohen_d_b2c3:.3f}")

```

```

=== Bloque 2 - Contraste 3: Mann-Whitney EUW vs KR ===

```

```

Mediana EUW = 3643.96 | Mediana KR = 3486.53
Media EUW = 3620.81 | Media KR = 3448.92
U = 2109362 | p-valor(bilateral) = 2.69e-08
r biserial de rangos = 0.1027

```

```

=== Contrastes complementarios paramétricos ===

```

```

Diferencia de medias EUW - KR = 171.89

```

```

Student var. iguales: t(3910) = 6.0048 | p-valor = 2.09e-09 | IC 95% = [115.77, 228.01]

```

```

Welch var. desiguales: t(3908.1) = 6.0046 | p-valor = 2.09e-09 | IC 95% = [115.77, 228.01]

```

```

Cohen's d = 0.192

```

El p-valor ≈ 0 nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que EUW y KR tengan la misma distribución de **damage_per_min**, es $\approx 0\%$.

Si H_0 fuese cierta, la probabilidad de observar una diferencia de rangos tan extrema como la obtenida sería prácticamente nula. Por tanto, rechazamos H_0 : hay evidencia de que la distribución de daño por minuto difiere entre EUW y KR.

En la muestra filtrada, EUW presenta una mediana de **damage_per_min** de 3643,96, frente a 3486,53 en KR. La diferencia va en sentido contrario a la creencia de la comunidad: en estos datos, EUW muestra valores algo superiores de daño por minuto.

El tamaño del efecto es pequeño ($r_{rb} = 0,103$). Por tanto, aunque la diferencia sea estadísticamente significativa, su magnitud es limitada.

Los contrastes paramétricos complementarios confirman la conclusión. La t de Student con varianzas iguales obtiene $t(3910) = 6,0048$, $p \approx 2,09 \cdot 10^{-9}$ e IC 95 % [115,77, 228,01]. La t de Welch, que no asume igualdad de varianzas, obtiene prácticamente el mismo resultado: $t(3908,1) = 6,0046$, $p \approx 2,09 \cdot 10^{-9}$ e IC 95 % [115,77, 228,01].

Esta coincidencia era esperable porque el test de Levene no detecta diferencias relevantes de dispersión y los tamaños muestrales son casi iguales. Por tanto, tanto el contraste no paramétrico como los dos contrastes paramétricos basados en el TCL apuntan en la misma dirección: EUW presenta mayor **damage_per_min** que KR en esta muestra.

El tamaño del efecto paramétrico también es pequeño ($d \approx 0,19$), coherente con el r_{rb} obtenido mediante Mann-Whitney.

7.4 Conclusión del Bloque 2

Los resultados muestran tres conclusiones diferenciadas. Primero, **kills** por equipo presenta sobredispersión y no se ajusta adecuadamente a una binomial negativa simple, lo que sugiere que la duración y el ritmo de partida son factores relevantes. Segundo, no se encuentra evidencia de dependencia entre clase de campeón y resultado, y la V de Cramér indica una asociación prácticamente nula. Tercero, sí se detecta una diferencia regional en **damage_per_min**, con valores superiores en EUW frente a KR en contra de lo que la comunidad cree, aunque con tamaño del efecto pequeño.

8 Bloque 3: Series Temporales (20%)

8.1 Serie temporal: duración media quincenal de las partidas

8.1.1 Planteamiento y contexto

Pregunta: ¿cómo evoluciona la duración media de las partidas a lo largo del periodo observado?

League of Legends es un juego que cambia por parches: ajustes de campeones, objetos, objetivos neutrales y ritmo de partida pueden modificar la duración media de las partidas a lo largo del tiempo.

En este apartado se plantea un análisis descriptivo de una serie temporal. Para ello utilizaremos filas de partida. Partimos de `df_team`, que contiene dos filas por partida, y reducimos a una observación por `match_id`. La variable temporal será `game_creation`, transformada a fecha, y la variable de interés será `match_duration_min`.

Antes de construir la serie filtraremos las partidas con duración inferior a 5 minutos. Estas partidas suelen corresponder a *remakes* o partidas anómalas, y no representan bien la duración real de una partida desarrollada con normalidad. Además, agruparemos las partidas en periodos de dos semanas para reducir la inestabilidad de semanas con muy pocas observaciones.

8.1.2 Hipótesis inicial

Contamos con una serie temporal:

- T_t : duración media de las partidas en el periodo quincenal t .

Se dispone de observaciones fechadas entre 2024 y 2026. Tras excluir partidas de duración inferior a 5 minutos, agregamos por periodos de dos semanas y calculamos:

$$\bar{T}_t = \frac{1}{n_t} \sum_{i=1}^{n_t} T_i$$

donde n_t es el número de partidas observadas en el periodo quincenal t .

En el código, T_i corresponde a `match_duration_min`, y el periodo quincenal se obtiene a partir de `game_creation`.

8.1.3 Preparación de la serie temporal

```
df_matches_ts = (
    df_team.sort_values("match_id")
    .drop_duplicates("match_id")
    .copy()
)
df_matches_ts["game_datetime"] = pd.to_datetime(df_matches_ts["game_creation"],
↪    unit="ms")
```

```

removed_short_b3 = int((df_matches_ts["match_duration_min"] < 5).sum())
df_matches_ts = df_matches_ts[df_matches_ts["match_duration_min"] >= 5].copy()

biweekly_duration = (
    df_matches_ts.set_index("game_datetime")
    .resample("2W-MON")
    .agg(
        n_matches=("match_id", "nunique"),
        duration_mean=("match_duration_min", "mean"),
        duration_median=("match_duration_min", "median"),
    )
    .dropna(subset=["duration_mean"])
    .reset_index()
    .rename(columns={"game_datetime": "period_start"})
)

low_n_periods_b3 = biweekly_duration.loc[
    biweekly_duration["n_matches"] < 5,
    ["period_start", "n_matches", "duration_mean", "duration_median"],
].copy()

print(f"Partidas únicas: {df_matches_ts['match_id'].nunique():,}")
print(f"Partidas eliminadas por duración < 5 min: {removed_short_b3:,}")
print(f"Primera fecha: {df_matches_ts['game_datetime'].min()}")
print(f"Última fecha: {df_matches_ts['game_datetime'].max()}")
print(f"Periodos quincenales observados:
↪ {biweekly_duration['period_start'].nunique():,}")
biweekly_duration[["n_matches", "duration_mean",
↪ "duration_median"]].describe().round(2)

```

Partidas únicas: 1,956

Partidas eliminadas por duración < 5 min: 44

Primera fecha: 2024-06-09 11:00:14.276000

Última fecha: 2026-01-16 17:06:36.918000

Periodos quincenales observados: 42

	n_matches	duration_mean	duration_median
count	42.00	42.00	42.00
mean	46.57	29.06	29.31
std	43.03	2.38	2.76
min	2.00	21.75	22.00
25%	18.00	27.93	28.12
50%	38.50	29.33	29.37
75%	71.50	30.33	30.30
max	229.00	37.22	37.22

```
fig, ax = plt.subplots(figsize=(7.0, 3.8))

ax.plot(
    biweekly_duration["period_start"],
    biweekly_duration["duration_mean"],
    color="steelblue",
    lw=1.4,
    marker="o",
    markersize=3,
    label="Media quincenal",
)

for i, period_start in enumerate(low_n_periods_b3["period_start"].head(2)):
    ax.axvline(
        period_start,
        color="gray",
        ls="--",
        lw=1.0,
        alpha=0.55,
        label="n < 5 partidas" if i == 0 else None,
    )

ax.set_title("Duración media quincenal de las partidas", fontsize=11)
ax.set_xlabel("Periodo quincenal", fontsize=10)
ax.set_ylabel("Duración media (min)", fontsize=10)
ax.tick_params(axis="x", labelrotation=35, labelsz=8)
ax.tick_params(axis="y", labelsz=9)
ax.legend(fontsize=8)

fig.tight_layout(pad=0.6)
plt.show()
```

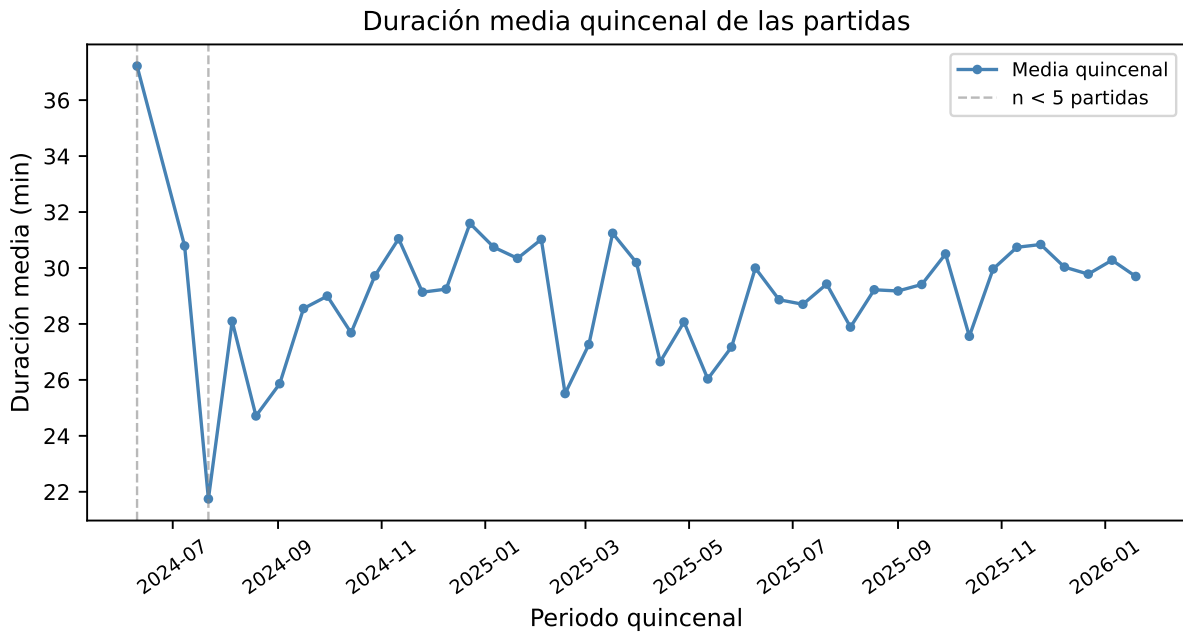


Figure 9: Evolución quincenal de la duración media de las partidas y periodos con bajo tamaño muestral.

8.1.4 Interpretación

La serie temporal permite observar la evolución de la duración media de las partidas a lo largo de los periodos de dos semanas disponibles. Podemos observar que al principio los valores están bastante dispersos, pero con el tiempo tienden a estabilizarse.

```
print("Periodos quincenales con menos de 5 partidas:")
low_n_periods_b3.head(2).round(2)
```

Periodos quincenales con menos de 5 partidas:

```
C:\Users\nicol\AppData\Local\Temp\ipykernel_10728\191902709.py:2: UserWarning: obj.round has
low_n_periods_b3.head(2).round(2)
```

	period_start	n_matches	duration_mean	duration_median
0	2024-06-10	2	37.22	37.22
2	2024-07-22	4	21.75	22.00

En estos periodos, una sola partida especialmente corta o larga puede desplazar mucho la media del periodo, provocando variabilidad en la serie temporal.

8.1.5 Modelo ARIMA: ajuste práctico y predicción

Intentaremos ajustar un modelo ARIMA sobre la duración media de dos semanas. Este modelo permite representar la dependencia temporal de la serie mediante componentes autorregresivos, diferenciación y medias móviles.

El objetivo es comprobar si una ARIMA sencilla captura la dinámica temporal observada y permite generar una predicción razonable a corto plazo.

La serie modelizada será:

$$Y_t = \text{duración media en el periodo } t$$

Un modelo ARIMA(p, d, q) combina:

- p : número de términos autorregresivos.
- d : número de diferenciaciones necesarias para estabilizar la serie.
- q : número de términos de media móvil.

Antes de seleccionar automáticamente el orden por AIC, conviene realizar un análisis previo de la serie:

- d se decide observando si la serie presenta tendencia o no estacionariedad. Si la serie ya oscila alrededor de un nivel estable, no sería necesario diferenciar ($d = 0$).
- p se relaciona con la PACF: *lags* iniciales significativos sugieren términos autorregresivos.
- q se relaciona con la ACF: autocorrelaciones iniciales significativas sugieren términos de media móvil.

8.1.6 Contraste de estacionariedad: Dickey-Fuller aumentado

Para justificar la elección de d , utilizaremos el contraste de Dickey-Fuller aumentado (ADF), que permite evaluar si una serie presenta raíz unitaria. La regresión auxiliar del ADF puede escribirse como:

$$\Delta Y_t = \alpha + \gamma Y_{t-1} + \sum_{i=1}^k \phi_i \Delta Y_{t-i} + \varepsilon_t$$

donde $\Delta Y_t = Y_t - Y_{t-1}$, k es el número de *lags* incluidos para corregir autocorrelación en los residuos y ε_t es un término de error. El contraste se plantea como:

- $H_0 : \gamma = 0$; la serie no es estacionaria.
- $H_A : \gamma < 0$; la serie es estacionaria.

El estadístico ADF se calcula como:

$$\tau_{\text{ADF}} = \frac{\hat{\gamma}}{\text{SE}(\hat{\gamma})}$$

En esta expresión, $\hat{\gamma}$ es la estimación muestral del coeficiente γ que acompaña a Y_{t-1} en la regresión auxiliar. Este coeficiente mide si el nivel anterior de la serie ayuda a explicar el cambio actual ΔY_t . $SE(\hat{\gamma})$ es el error estándar de esa estimación. El cociente τ_{ADF} indica cuántos errores estándar se aleja $\hat{\gamma}$ de 0. Si τ_{ADF} es negativo y significativo, rechazamos H_0 y consideramos que la serie es estacionaria.

```
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

biweekly_arima =
    ↪ biweekly_duration.set_index("period_start")["duration_mean"].sort_index()
biweekly_index = pd.date_range(
    biweekly_arima.index.min(),
    biweekly_arima.index.max(),
    freq="2W-MON",
)

y_arima = (
    biweekly_arima
    .reindex(biweekly_index)
    .interpolate(method="time")
    .ffill()
    .bfill()
)

y_arima.name = "duration_mean"
y_arima_diff = y_arima.diff().dropna()

adf_level = adfuller(y_arima, autolag="AIC")
adf_diff = adfuller(y_arima_diff, autolag="AIC")

adf_table = pd.DataFrame(
    {
        "serie": ["Nivel", "Primera diferencia"],
        "estadistico_ADF": [adf_level[0], adf_diff[0]],
        "p_valor": [adf_level[1], adf_diff[1]],
        "lags_usados": [adf_level[2], adf_diff[2]],
    }
)

print("=== Diagnóstico previo para ARIMA ===")
print(f"Periodos quincenales modelizados: {len(y_arima):,}")
print("Test ADF de estacionariedad:")
print(adf_table.round(4).to_string(index=False))
```

```
=== Diagnóstico previo para ARIMA ===
Periodos quincenales modelizados: 43
Test ADF de estacionariedad:
      serie  estadistico_ADF  p_valor  lags_usados
```

Nivel	-5.3589	0.0	0
Primera diferencia	-6.1364	0.0	4

```
fig, axes = plt.subplots(2, 1, figsize=(7.0, 3.8), sharex=True)

axes[0].plot(y_arima.index, y_arima, color="steelblue", lw=1.2)
axes[0].set_title("Duración media quincenal (nivel)", fontsize=11)
axes[0].set_ylabel("Minutos", fontsize=10)
axes[0].tick_params(axis="y", labelsizes=9)

axes[1].plot(y_arima_diff.index, y_arima_diff, color="crimson", lw=1.2)
axes[1].axhline(0, color="gray", ls=":", lw=1.1)
axes[1].set_title("Primera diferencia", fontsize=11)
axes[1].set_xlabel("Periodo quincenal", fontsize=10)
axes[1].set_ylabel("Cambio", fontsize=10)
axes[1].tick_params(axis="x", labelrotation=35, labelsizes=8)
axes[1].tick_params(axis="y", labelsizes=9)

fig.tight_layout(pad=0.6)
plt.show()
```

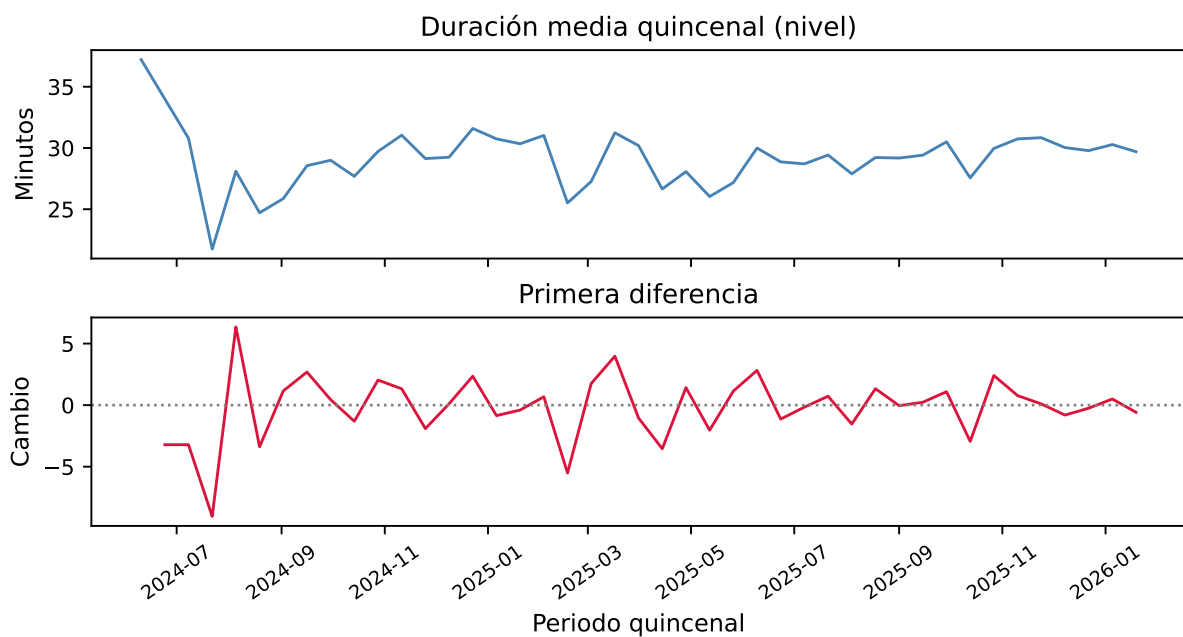


Figure 10: Serie quincenal en niveles y primera diferencia.

```
fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.8))

plot_acf(y_arima, lags=12, ax=axes[0], zero=False)
axes[0].set_title("ACF", fontsize=11)
axes[0].tick_params(labelsizes=9)
```

```

plot_pacf(y_arima, lags=12, ax=axes[1], zero=False, method="ywm")
axes[1].set_title("PACF", fontsize=11)
axes[1].tick_params(labelsize=9)

fig.tight_layout(pad=0.6)
plt.show()

```

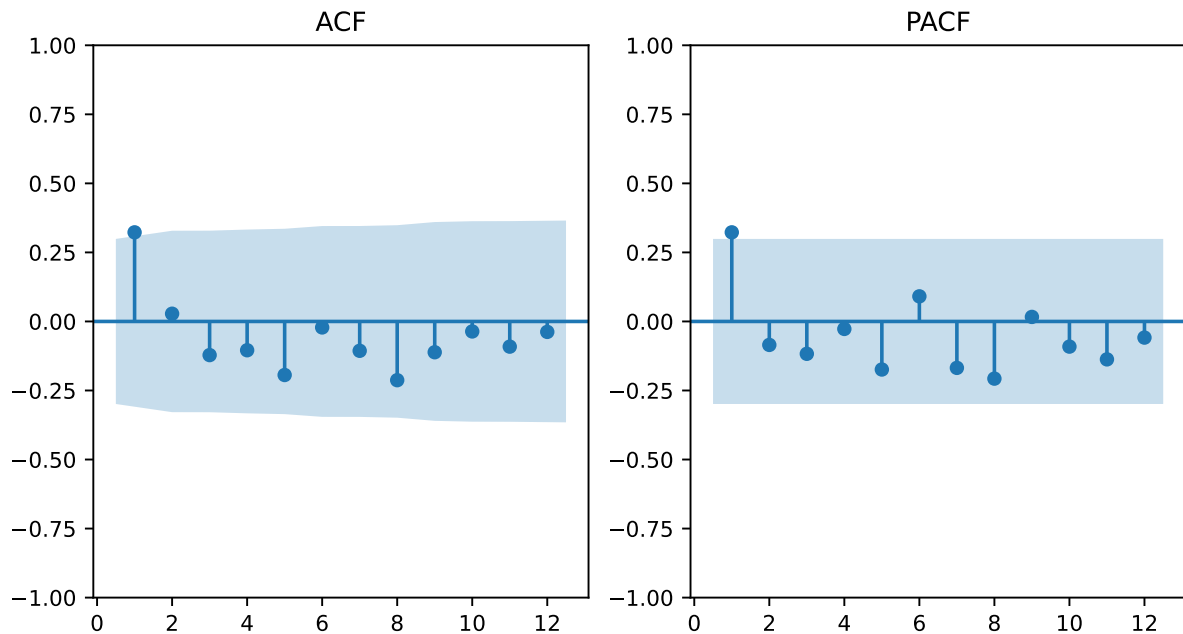


Figure 11: Correlogramas ACF y PACF de la serie quincenal.

Visualmente, la serie quincenal no muestra una tendencia creciente o decreciente clara, sino oscilaciones alrededor de un nivel relativamente estable. Esto se contrasta mediante el ADF.

El estadístico observado es $\tau_{ADF} = -5,3589$ y el p-valor asociado es $p \approx 4,13 \cdot 10^{-6}$.

El p-valor de $p \approx 4,13 \cdot 10^{-6}$ nos indica que el grado de compatibilidad de la muestra con la hipótesis nula, es decir, que la serie no es estacionaria, es $\approx 0\%$.

Si H_0 fuese cierta, la probabilidad de obtener un valor del estadístico de contraste más extremo que $\tau_{ADF} = -5,3589$ sería $\approx 0\%$. Por tanto, rechazamos $H_0 : \gamma = 0$: hay evidencia a favor de que la serie es estacionaria.

En términos del modelo ARIMA, esto justifica tomar inicialmente $d = 0$.

En los correlogramas podemos observar que tanto la PACF como la ACF concentran la señal en los primeros *lags*, lo que justifica pensar en valores bajos de p y q (1-2).

Seleccionaremos finalmente el orden (p, d, q) mediante el criterio AIC, escogiendo el modelo con menor valor. Esta búsqueda servirá para comprobar si la lectura visual de la serie y de los correlogramas coincide con el modelo que mejor ajusta según este criterio.

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox
import warnings

biweekly_arima =
    ↪ biweekly_duration.set_index("period_start")["duration_mean"].sort_index()
biweekly_index = pd.date_range(
    biweekly_arima.index.min(),
    biweekly_arima.index.max(),
    freq="2W-MON",
)

y_arima = (
    biweekly_arima
    .reindex(biweekly_index)
    .interpolate(method="time")
    .ffill()
    .bfill()
)
y_arima.name = "duration_mean"

candidate_rows = []
best_aic = np.inf
best_order = None
best_model = None

with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    for p in range(0, 4):
        for d in range(0, 3):
            for q in range(0, 4):
                if p == 0 and d == 0 and q == 0:
                    continue
                try:
                    model = ARIMA(
                        y_arima,
                        order=(p, d, q),
                        enforce_stationarity=False,
                        enforce_invertibility=False,
                    ).fit()
                    candidate_rows.append(
                        {
                            "p": p,
                            "d": d,
                            "q": q,
                            "AIC": model.aic,
                            "BIC": model.bic,
                        }
                    )
```

```

        )
        if model.aic < best_aic:
            best_aic = model.aic
            best_order = (p, d, q)
            best_model = model
    except Exception:
        continue

arima_candidates = pd.DataFrame(candidate_rows).sort_values("AIC")
ljung_box = acorr_ljungbox(best_model.resid.dropna(), lags=[8], return_df=True)

print("=== Modelo ARIMA sobre duración media quincenal ===")
print(f"Periodos quincenales modelizados: {len(y_arima):,}")
print(f"Orden seleccionado por AIC: ARIMA{best_order}")
print(f"AIC = {best_model.aic:.2f} | BIC = {best_model.bic:.2f}")
print("Mejores modelos candidatos:")
print(arima_candidates.head(8).round(2).to_string(index=False))
print("Ljung-Box sobre residuos (lag 8):")
print(ljung_box.round(4).to_string())

```

```

=== Modelo ARIMA sobre duración media quincenal ===
Periodos quincenales modelizados: 43
Orden seleccionado por AIC: ARIMA(2, 0, 3)
AIC = 150.09 | BIC = 161.73
Mejores modelos candidatos:
  p  d  q   AIC   BIC
2  0  3 150.09 161.73
0  1  3 150.32 156.88
2  1  3 151.89 161.72
1  1  3 152.04 160.23
3  0  3 152.14 165.44
0  0  3 152.24 160.56
3  1  3 152.97 164.44
1  0  3 154.17 164.15
Ljung-Box sobre residuos (lag 8):
   lb_stat  lb_pvalue
8    5.9815    0.6493

```

```

fitted_arima = best_model.fittedvalues

fig, ax = plt.subplots(figsize=(7.0, 3.8))

ax.plot(
    y_arima.index,
    y_arima,
    color="steelblue",
    lw=1.2,

```

```

    label="Serie observada",
)
ax.plot(
    fitted_arima.index,
    fitted_arima,
    color="crimson",
    lw=1.5,
    alpha=0.85,
    label=f"Ajuste ARIMA{best_order}",
)

ax.set_title("Duración media quincenal: observado vs ARIMA ajustado",
    ↪  fontsize=11)
ax.set_xlabel("Periodo quincenal", fontsize=10)
ax.set_ylabel("Duración media (min)", fontsize=10)
ax.tick_params(axis="x", labelrotation=35, labelsize=8)
ax.tick_params(axis="y", labelsize=9)
ax.legend(fontsize=8)

fig.tight_layout(pad=0.6)
plt.show()

```

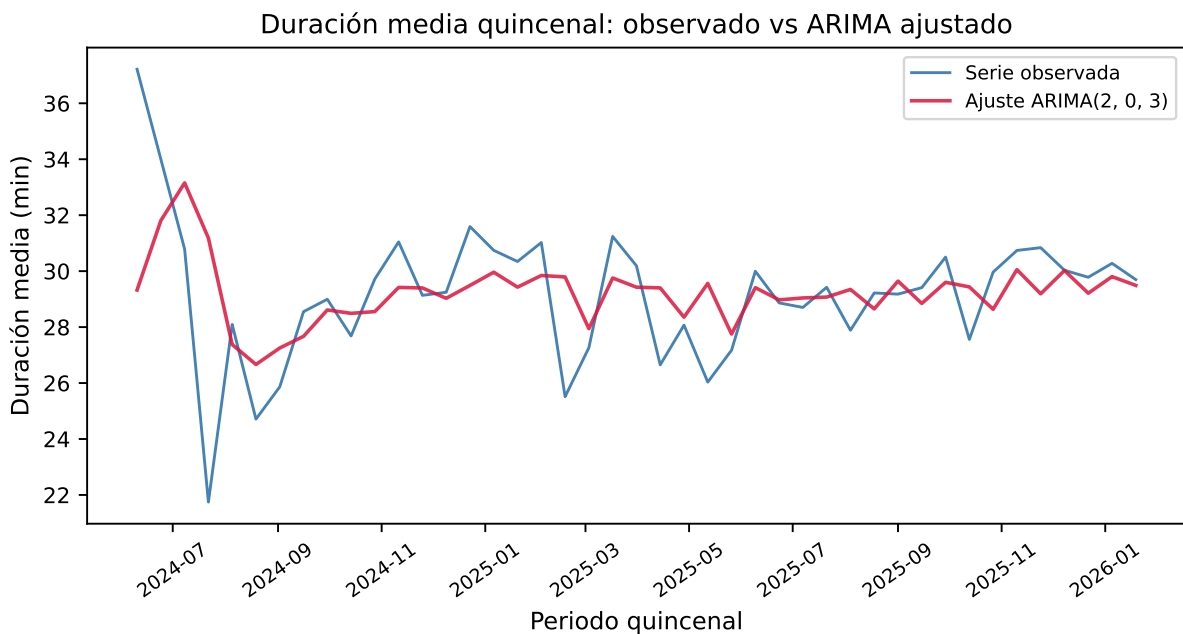


Figure 12: Ajuste ARIMA sobre la duración media quincenal.

Una vez ajustado el modelo, generamos una predicción a dos periodos hacia delante, equivalente aproximadamente a cuatro semanas. Esta predicción debe interpretarse como una extrapolación local del patrón reciente, no como una previsión fuerte del metajuego futuro.

```

forecast_steps = 2
forecast_arima = best_model.get_forecast(steps=forecast_steps)
forecast_mean = forecast_arima.predicted_mean
forecast_ci = forecast_arima.conf_int()

forecast_table = pd.DataFrame(
    {
        "prediccion": forecast_mean,
        "IC_low": forecast_ci.iloc[:, 0],
        "IC_high": forecast_ci.iloc[:, 1],
    }
)

print("=== Predicción ARIMA a 2 periodos quincenales ===")
print(forecast_table.round(2).to_string())

```

```

=== Predicción ARIMA a 2 periodos quincenales ===
          prediccion  IC_low  IC_high
2026-02-02        29.53   26.84   32.22
2026-02-16        29.43   26.57   32.28

```

```

fig, ax = plt.subplots(figsize=(7.0, 3.8))

ax.plot(
    y_arima.index,
    y_arima,
    color="steelblue",
    lw=1.2,
    label="Serie observada",
)
ax.plot(
    forecast_table.index,
    forecast_table["prediccion"],
    color="crimson",
    lw=1.8,
    marker="o",
    label="Predicción ARIMA",
)
ax.fill_between(
    forecast_table.index,
    forecast_table["IC_low"],
    forecast_table["IC_high"],
    color="crimson",
    alpha=0.18,
    label="IC 95% predicción",
)

```

```

ax.set_title("Predicción de duración media quincenal", fontsize=11)
ax.set_xlabel("Periodo quincenal", fontsize=10)
ax.set_ylabel("Duración media (min)", fontsize=10)
ax.tick_params(axis="x", labelrotation=35, labelsize=8)
ax.tick_params(axis="y", labelsize=9)
ax.legend(fontsize=8)

fig.tight_layout(pad=0.6)
plt.show()

```

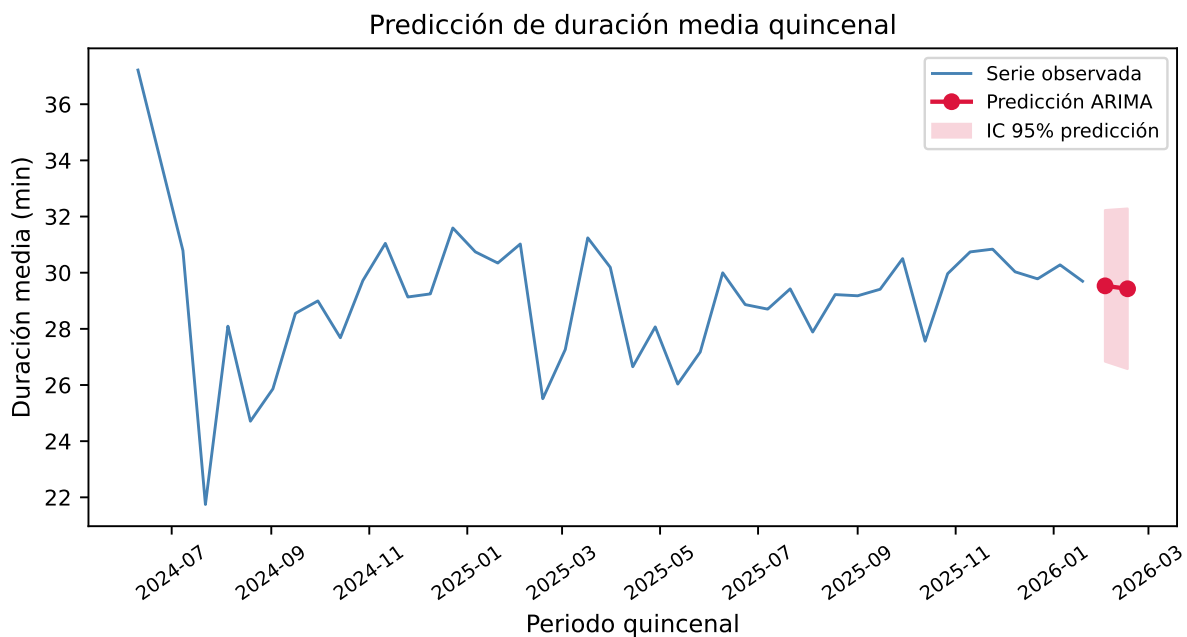


Figure 13: Predicción ARIMA a dos periodos quincenales de la duración media.

El modelo seleccionado por AIC es un $\text{ARIMA}(2, 0, 3)$, con $\text{AIC} = 150,09$.

Esta elección es coherente con el diagnóstico previo. La serie no mostraba una tendencia clara y el test ADF rechazaba la no estacionariedad, por lo que $d = 0$ era una opción razonable antes de este contraste. Además, los correlogramas sugerían dependencia concentrada en *lags* bajos, lo que justifica los valores de p y q seleccionados.

En la tabla de modelos candidatos se observa que el BIC favorece ligeramente un $\text{ARIMA}(0, 1, 3)$, con $\text{BIC} = 156,88$. BIC penaliza más la complejidad que el AIC y suele preferir modelos más simples. En nuestro caso, priorizamos el AIC porque buscamos un ajuste razonable para una predicción exploratoria a corto plazo. Además, no elegimos el modelo con $d = 1$ solo por tener mejor BIC, ya que el contraste ADF ya indicaba que la serie en niveles era estacionaria.

El test de Ljung-Box se aplica sobre los residuos hasta el *lag* 8. Con esto comprobamos si queda autocorrelación residual con los últimos 4 meses. El p-valor obtenido es 0,65, por lo que no se detecta autocorrelación residual significativa. Esto sugiere que el modelo captura razonablemente la dependencia temporal de la serie.

La predicción a dos periodos de dos semanas sitúa la duración media esperada en torno a 29-30 minutos, con intervalos moderadamente amplios. En concreto, las predicciones son aproximadamente 29,53 y 29,43 minutos, mientras que los intervalos al 95% abarcan rangos cercanos a 26,5-32,3 minutos.

El ARIMA no apunta a un cambio brusco en la duración media de las partidas, sino a una continuidad de la duración reciente. La amplitud de los intervalos refleja la incertidumbre propia de la muestra y la variabilidad temporal. Además, en *League of Legends* cualquier parche futuro puede alterar la dinámica de la serie, por lo que esta predicción debe entenderse como un ejercicio y no como una predicción robusta.

9 Conclusiones

El objetivo de este trabajo era aplicar técnicas de inferencia estadística a un caso real de *League of Legends*, intentando comprobar con datos algunas ideas que suelen aparecer en la comunidad: la importancia del oro temprano, la posible ventaja del lado azul, las diferencias entre roles, clases de campeón y regiones, y la evolución temporal de la duración de las partidas.

En el Bloque 1, el resultado más claro es la importancia de la ventaja de oro al minuto 15. La diferencia media entre el equipo ganador y el perdedor es positiva, estadísticamente significativa y con un tamaño del efecto relevante. Esto encaja con la intuición del juego: una ventaja temprana de oro suele traducirse en mejores objetos, más control del mapa y mayor capacidad de cerrar la partida. Aun así, también se ha visto que no es una condición suficiente para ganar, ya que hay partidas en las que el equipo con menos oro al minuto 15 consigue remontar. En cambio, no se ha encontrado evidencia suficiente para afirmar que el lado azul tenga una probabilidad de victoria superior al 50%, y tampoco se observan diferencias prácticas relevantes entre ADC y Mid en daño total infligido, por lo que su capacidad de liderar al equipo en la partida es similar.

En el Bloque 2, los métodos no paramétricos han permitido completar el análisis desde otra perspectiva. El contraste de bondad de ajuste muestra que las `kills` por equipo no se ajustan adecuadamente a una binomial negativa. Esto tiene sentido en el contexto del juego, porque los asesinatos dependen de muchos factores que no están en una distribución simple: duración, ritmo, composición, objetivos y estado de la partida. También se ha visto que no hay evidencia clara de dependencia entre clase de campeón y resultado, con una *V* de Cramér prácticamente nula. Por último, sí aparece una diferencia regional en `damage_per_min`, con valores superiores en EUW frente a KR, contrario a lo que se podría esperar, ya que en la comunidad se suele pensar que en Corea los jugadores son más agresivos. Sin embargo, el tamaño del efecto es pequeño, por lo que debe interpretarse con cuidado.

En el Bloque 3, la serie temporal ha servido para analizar la duración media de las partidas a lo largo del tiempo. Se filtraron las partidas de menos de 5 minutos por considerarse *remakes* o partidas anómalas, y se agrupó la duración en periodos de dos semanas para reducir el ruido de semanas con pocas observaciones. El análisis ADF indicó que la serie quincenal era estacionaria, por lo que no fue necesario diferenciarla ($d = 0$). A partir de los correlogramas y la selección por AIC se ajustó un ARIMA(2, 0, 3). El modelo no apunta a un cambio brusco inmediato en la duración media, sino a una continuidad cercana a los 29-30 minutos.

En conjunto, el trabajo muestra que algunas creencias habituales sí encuentran respaldo estadístico, especialmente la relevancia del oro temprano, mientras que otras no aparecen con tanta fuerza

en esta muestra, como la ventaja del lado azul o la influencia directa de la clase de campeón sobre el resultado. También se observa que encontrar significación estadística no siempre implica encontrar un efecto grande o importante en la práctica, como ocurre con la comparación regional de daño por minuto.

Las principales limitaciones vienen del propio diseño de los datos. La muestra depende de jugadores semilla y de la disponibilidad de la API, por lo que no puede considerarse una muestra perfectamente aleatoria de todas las partidas de SoloQ. Además, algunas observaciones no son completamente independientes, ya que un mismo jugador puede aparecer en varias partidas, y el metajuego cambia con los parches. Tampoco se controlan variables relevantes como composición de campeones, objetivos neutrales, nivel exacto de los jugadores o estado de la partida en minutos concretos.

Como líneas futuras, sería interesante ampliar el dataset, separar los análisis por parche o por nivel competitivo, e incorporar variables más detalladas de la partida, como dragones, torres, barones, experiencia o ventaja de oro total por minuto. También se podrían plantear modelos más completos, pero manteniendo el enfoque inferencial. Nuestro objetivo es entender qué variables aportan evidencia real y qué magnitud tienen dentro del juego, no simplemente predecir quién gana.

10 Bibliografía

1. Do, T. D., Wang, S. I., Yu, D. S., McMillian, M. G., & McMahan, R. P. (2021). *Using Machine Learning to Predict Game Outcomes Based on Player-Champion Experience in League of Legends* (FDG 2021). <https://arxiv.org/abs/2108.02799>
2. Jack J. (2022). *How Much Does Side Selection Matter, Really?* The Esports Analyst Club. <https://medium.com/the-esports-analyst-club-by-itero-gaming/how-much-does-side-selection-matter-really-59bfc17b93de>
3. Lin, L. (2016). *League of Legends Match Outcome Prediction* (CS229 Project Report, Stanford). <https://cs229.stanford.edu/proj2016/report/Lin-LeagueOfLegendsMatchOutcomePrediction-report.pdf>
4. Payne, E. H., Hardin, J. W., Egede, L. E., Ramakrishnan, V., Selassie, A., & Gebregziabher, M. (2017). Approaches for dealing with various sources of overdispersion in modeling count data: Scale adjustment versus modeling. *Statistical Methods in Medical Research*, 26(4), 1802-1823. <https://doi.org/10.1177/0962280215588569>
5. Riot Games. (s. f.). *Riot Developer Portal — API Methods (Match-V5)*. <https://developer.riotgames.com/api-methods/>